

Szegedi Tudományegyetem
Informatikai Intézet

SZAKDOLGOZAT

Tóth Bence

2026

**Szegedi Tudományegyetem
Informatikai Intézet**

**Könyvek életútjának nyomon követése blokklánc
technológiával, QR-kód alapú azonosítással és
gamifikációs elemekkel**

**Tracking the Lifecycle of Books Using Blockchain Technology, QR
Code-Based Identification, and Gamification Elements**

Szakdolgozat

Készítette:

Tóth Bence

informatika szakos hallgató

Témavezető:

Dr. Pflanzner Tamás

egyetemi adjunktus

**Szeged
2026**

Feladatkiírás

A szakdolgozat célja egy olyan webes alkalmazás prototípusának megvalósítása, amely könyvek életútját követi nyomon. A könyvekhez tartozó események, például birtoklás, olvasási megjegyzések vagy állapotváltozások blokkláncon rögzített vagy ellenőrzött módon kerülnek rögzítésre. A rendszer QR-kód alapú azonosítást alkalmaz, amely lehetővé teszi a könyvek fizikai és digitális összekapcsolását, valamint gamifikációs elemekkel ösztönzi a felhasználói aktivitást.

Tartalmi összefoglaló

- **A téma megnevezése:**

Web alapú, blokklánccal hitelesített könyv életút követő alkalmazás

- **A megadott feladat megfogalmazása:**

A feladat egy olyan webalkalmazás elkészítése, amely felhasználói felülete egyszerű, lehetőséget biztosít könyvek hozzáadására. Ezt követően értékelési lehetőséget biztosít a felhasználónak, majd továbbadhatja az azonosító könyvjelzővel ellátott könyvet, amivel a következő tulajdonos beolvasást követően szintén tud értékelni, és láthatja korábbi tulajdonosai értékeléseit is.

- **A megoldási mód:**

A munkafolyamat egy részletes programterv elkészítésével kezdődött, a képernyőtervek és az adatbázis táblák egy erős keretet adtak ami jelentősen felgyorsította a fejlesztést. A program megírását a frontend oldalról kezdtem, majd alkalmaztam az adatbázis terveket a *PostgreSQL* adatbázisban. Amikor a teljes adatbázis készen állt, kialakítottam a *Node.js* alapú backend rendszert, ezt követően összekapcsoltam az *Angular* frontendet az elkészített endpointokkal, végül pedig a *Solidity* okosszerződést készítettem el, amit tesztelés és deploy után hozzáfűztem a már működő webalkalmazáshoz.

- **Alkalmazott eszközök, módszerek:**

A webalkalmazás frontendjéhez az *Angular* keretrendszert használtam a *PrimeNG* komponens könyvtár társaságában. A backend egy *Node.js* alapokon fekvő *REST API*, ami a *PostgreSQL* adatbázisban tárolt adatokat használja fel. A Sepolia Ethereum testnetre depolyolt okosszerződést *Hardhat* fejlesztőkörnyezetben készítettem el, az *Ethers.js* könyvtárat alkalmazva.

- **Elért eredmények:**

Az alkalmazás a modern irányelveknek megfelelő, intuitív kezelőfelület és a blokklánc által biztosított adatbiztonság eredményeként egy könnyen használható, de funkcionális program melyben a tervezett funkciók többsége rendeltetésszerűen működik.

- **Kulcsszavak:**

PostgreSQL, Node.js, REST API, Hardhat, Ethers.js

Tartalomjegyzék

Feladatkiírás.....	3
Tartalmi összefoglaló.....	4
Tartalomjegyzék.....	5
2. Kapcsolódó technológiák és elméleti háttér.....	7
2.1 Blokklánc alapjai.....	7
2.2 Ethereum és okoszerződések.....	8
2.3 EIP-712 aláírás.....	9
2.4 ERC-1155 token standard.....	11
2.5 QR-kód alapú azonosítás.....	12
2.6 Gamifikáció alapjai.....	13
3. Követelményanalízis.....	14
3.1 Funkcionális követelmények.....	14
3.2 Nem funkcionális követelmények.....	17
3.3 Felhasználási esetek.....	18
4. Rendszerterv.....	19
4.1 Architektúra áttekintés.....	19
4.2 Adatbázis tervezés.....	20
4.3 Blockchain architektúra.....	22
4.4 Okoszerződés tervezése.....	22
4.5 Backend működése.....	25
4.6 Frontend működése.....	26
5. Fejlesztés során felmerülő problémák és megoldások.....	28
5.1 Blokklánc tranzakciók szinkronizációja.....	28
5.2 EIP-712 aláírás implementációja.....	29
5.3 Fájlfeltöltés és médiakezelés.....	30
5.4 Frontend és wallet integráció.....	31
5.5 Könyv és esemény egyidejű létrehozása.....	32
6. Tesztelés és kiértékelés.....	33
6.1 Tesztkörnyezet.....	33
6.2 Funkcionális tesztek.....	34
6.3 Backend stresszteszt.....	35
6.4 Tranzakciós költségek és blokklánc korlátok.....	37
7. Eredmények és értékelés.....	39
8. Továbbfejlesztési lehetőségek.....	40
9. Összefoglalás.....	41
Irodalomjegyzék.....	43
Nyilatkozat.....	46

1. Bevezetés

A modern informatikai rendszerek jelentős része centralizált működésű, amely számos problémát vet fel az adatok megbízhatósága és átláthatósága szempontjából. Az ilyen rendszerekben tárolt információk módosíthatósága miatt nehézkes az események hiteles visszakövetése. Ez a probléma fizikai tárgyak esetében is jelen van, például a személyjármű-piacon található hirdetések nagy része pontatlan vagy félrevezető információkat tartalmaz az adott gépjárműről a könnyebb eladás érdekében. Erre készült megoldásként a Carvertical [1] szolgáltatás, amely részben blokklánc alapú megoldásokat is alkalmaz az adatok hitelességének növelésére, ezzel csökkenti járművásárlás során a kockázatokat.

Az olvasók körében is észrevehető ez a probléma. Ugyan nem a könyv minősége van szempontban, hanem az érzelmi kötődés egy adott példánnyal. A legtöbb könyvimádó szereti megtartani a könyveket, amelyeket elolvasott, mivel emiatt bármikor eszébe juthatnak az olvasás közben keletkezett élmények. Nem mindenki engedheti meg magának, akár anyagi helyzet vagy helyhiány miatt, hogy megtartson minden könyvet.

A dolgozat célja egy olyan webalapú rendszer megvalósítása, amely blokklánc technológia, QR-kódos azonosítás [2] és gamifikációs elemek [3] alkalmazásával lehetővé teszi a könyvek életútjának követését. A webalkalmazás megoldást nyújt azon olvasóknak, akik szeretnék megtartani könyveiket emlékként, de fizikailag nincs módjukban ezt megtenni. Hozzáadhatják könyveiket képekkel, értékeléssel társítva az adatbázishoz, ahol az adatok módosítása korlátozott és visszakövethető, a blokklánc decentralizált megvalósítása [4] miatt.

A rendszer nemcsak a könyvet feltöltő felhasználó számára kínál előnyöket. A könyv következő tulajdonosa egy egyedi grafikával ellátott könyvjelző segítségével jut hozzá az adott példányhoz, amely tartalmazza a könyvhöz tartozó QR-kódot. A kód beolvasásával megtekinthetők az előző tulajdonosok által hozzáadott képek és értékelések, valamint lehetőség nyílik új tartalmak rögzítésére is, így a könyv története folyamatosan bővül.

2. Kapcsolódó technológiák és elméleti háttér

A fejezet célja a webalkalmazás fejlesztése során alkalmazott főbb technológiák és a hozzájuk kapcsolódó elméleti alapok bemutatása. A rendszer megvalósítása során több modern webfejlesztési technológia kerül felhasználásra, különös tekintettel a blokklánc alapú adathitelesítésre, amely a rendszerben rögzített adatok visszakövethetőségének biztosításában játszik kulcsfontosságú szerepet. Emellett az azonosítást támogató QR-kód alapú megoldások is bemutatásra kerülnek.

2.1 Blokklánc alapjai

A blokklánc egy decentralizált adatkezelési technológia, amely biztonságosan és nehezen módosíthatóan rögzíti az adatokat összekapcsolt csomópontok hálózatán. Kifejlesztésének elsősorban egy megbízhatóbb pénzügyi platform létrejötte volt a cél, ugyan, számos iparágban megtalálta helyét a technológia. A tranzakciók adatblokkonként élnek összekapcsolva amelyek alkotják a láncot, így a már egyszer hozzákapcsolt adatok gyakorlatilag nem módosíthatók.

A decentralizáció definíciója szerint az adatokat nem egy központi adatbázis szerver tárolja, hanem egy számítógép-hálózat tartja fenn. Ebből adódik a blokklánc legfontosabb tulajdonsága, az adatintegritás és biztonság, ugyanis, ha az egyik csomóponton módosulna az adat, az összes többin pedig az eredeti tartalom szerepel, ez eltérést jelez a hálózat állapotában.

Egy blokk felépítése a tranzakciók adataiból, egy egyedi hash kódból, valamint az előző blokk hash értékéből és egy időbélyegből áll. Így kapcsolódnak a blokkok egymáshoz, láncot alkotva. Az egyik legfontosabb eleme a kriptográfiai azonosítás [5], mivel, ha egy blokk értéke megváltozik, az módosítja a hash értékét is, így a következő blokk továbbra is a régi hash értékre hivatkozik, emiatt a lánc érvénytelenné válik.

A hash egyfajta digitális ujjlenyomat, amit egy speciális matematikai függvény hoz létre tetszőleges adatokból, meghatározott hosszú karakterlánc formájában. Determinisztikus mivolta miatt egy adott bemenetnek mindig azonos hash értéke lesz, viszont akármilyen kis változás a kapott adatban drasztikusan átalakítja a kimeneti karakterláncot. A hashelés egy egyirányú művelet, szinte lehetetlen helyreállítani belőle az eredeti adatokat, és arra, hogy két különböző bemenetnek azonos hash értéke legyen, az esély nullához közeli.

A tranzakció nem feltétlen azonos a köznyelvben használt tranzakció definíciójával, ugyanis a blokkláncon minden adatátvitel vagy utalás, amit a hálózat résztvevői azonosítanak és érvényesítenek, tranzakciónak nevezhető. Nem egyértelmű valutamozgásról van szó, hanem egy digitális aláírással hitelesített bejegyzésről, amely megváltoztatja a lánc felépítését. Ebbe beletartoznak a kriptovaluta küldések, fogadások, az okosszerződések végrehajtása, az adatok rögzítése és az NFT-ekkel [6] kapcsolatos műveletek is. A hálózatot alakító csomópontok validálják, vagy bányásszák a tranzakciókat, ezzel ellenőrizve az érvényességüket. Minden tranzakció hálózati díjat von maga után, például Ethereum [7] esetében gázdíjat, amit a feldolgozó validátorok kapnak.

A dolgozat eredményeként elkészült alkalmazásban minden könyv és könyvesemény hozzáadása egy tranzakció, aminek eredményeképp a hozzáadott adatok hash értéke kerül láncra rögzítésre. A hozzáadáskor validált adatok az eredeti adatok, és ha a centralizált PostgreSQL [8] adatbázisban is eltárolt adatok módosulnak, azonnal észrevehető, hogy a hash értékek különböznek, emiatt hitelességét veszti az adat.

2.2 Ethereum és okosszerződések

Az Ethereum egy nyílt forráskódú decentralizált blokklánc technológia. A natív valutája az Ether (ETH), 2015 óta van jelen a kriptovaluta közösségben. Fő funkcionalitása az okosszerződésekben [9] rejlik, emiatt nem csak adattárolásra használható, hanem programok futtatására is. Az okosszerződések készítői a Solidity [10] programozási nyelven fejlesztik programjaikat amelyek az Ethereum Virtual Machine-en (EVM) [11] futnak. Az EVM egy virtuális számítógép, melyben a számításokat csomópontok elosztott hálózata végzi.

Az okosszerződés egy blokkláncon élő, feltételekből álló programkód, ami automatikus végrehajtást biztosít feltételek teljesülése alapján. Az említett kódok a láncra való közzététel után átláthatóak és gyakorlatilag nem módosíthatók. A csupán olvasást végző műveletek nem vonnak anyagi költséget maguk után, viszont az adatokat módosító cselekvések úgynevezett gas díj fizetésével járnak. A könyv-történet kezelő webalkalmazásban egy Solidity nyelven íródott okosszerződés működik. A szerződés biztosítja a könyv, esemény adatok integritásának biztosítását, ezen adatok hash

értékének eltárolásával, valamint a felhasználói aláírások kezelése mellett a gamifikációt megtestesítő kitűző rendszer működéséért is a programkód felel.

Az Ethereum hálózaton végrehajtott műveletek költségét nevezzük gas-nak. A szükségét a validálást végző validátorok megfizetése indokolja, a hálózatban dolgozó csomópontok nagy erőforrásokat igénylő számításokat végeznek, ezért kapják fizetségül a gas díjat [12]. A tranzakciós költség mennyisége a művelet bonyolultságával azonosan változik. A diplomamunkában minden könyv és esemény hozzáadása során gas díj kerül kifizetésre amelyet a backend által vezérelt fiók fizet, a felhasználói kényelem érdekében. Emiatt játszanak különösen fontos szerepet a hash azonosítók, ugyanis, ha minden könyv valamennyi adata a láncon lenne tárolva, az jelentős gas díj emelkedést jelentene.

A webalkalmazáshoz legjobban használható blokklánc technológia kiválasztása során több megoldás közül volt lehetőségem választani. A Bitcoin [13], ami a legismertebb blokklánc alapú rendszer, főként a pénzügyi oldalon emelkedik ki, korlátozottan támogat okosszerződések futtatását, emiatt nem találtam alkalmasnak az ötlet megvalósítására. A különböző blokkláncok közül a modernebb Solana [14] egy gyorsabb és olcsóbb működtetést ígér, viszont kissé centralizáltabb, mint az Ethereum, vagy a Polygon [15] ami hasonló indíttatással jött létre mint a Solana, de eltérő fejlesztési megközelítést igényel. Mindent összevetve, az Ethereum hatalmas közössége, dokumentációja, számos fejlesztői csomag és eszköz biztosítása egyértelmű választás volt.

2.3 EIP-712 aláírás

Az Ethereum hálózaton használt digitális aláírás alapvető biztonsági funkció, amely során az aláíró fél a privát kulcsával hitelesíti a tranzakciókat vagy egyéb üzeneteket. Az aláírás biztosítja, hogy valóban a tárca tulajdonosa kezdeményezte a tranzakciót, és nem egy harmadik fél, vagy támadó. A privát kulcsos működéséből adódóan számos web3-as alkalmazás hasznosítja jelszó nélküli beléptetéshez, mivel alkalmas személyazonosság bizonyítására.

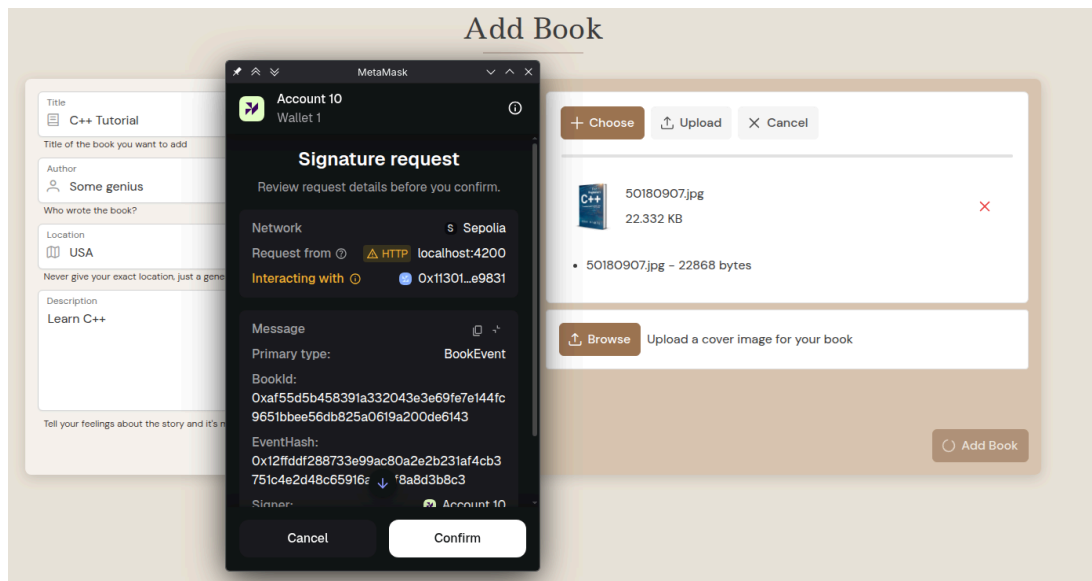
A hagyományos, strukturálatlan aláírási módszerek (pl. EIP-191), kevésbé felhasználóbarátak. Legfőbb hátrányuk az egyetlen hexadecimális kód amely első ránézésre érthetetlen, különböző eszközök és időráfordítás szükséges ennek

értelmezéséhez. Emiatt jóval gyakoribbak az átverések és visszaélések ilyen aláírásokat használó rendszereknél. Az aláírások önmagukban nem igényelnek gas díjat, viszont az EIP-191 [16] szabvány korlátozottan, nehezen használható gas díj nélküli tranzakciókra.

Az EIP-712 aláírás kialakításakor ezeket a hátrányokat vették alapul, majd megoldást kerestek ezekre. Az új szabvány már strukturált, olvasható formában jeleníti meg az adatokat a wallet felületén, ahol megtalálható a küldő és fogadó fél címe, és a tranzakcióban szereplő adatok. Ezáltal lényegesen egyszerűbb gyorsan áttekinteni az aláírást, ezzel a csalásokat megelőzve. Az EIP-712 [16] alkalmas metatranzakciók megvalósításának támogatására, így egyértelmű választássá alakult az aláírások lebonyolításához.

Az EIP-712 mint szabvány egy strukturált adatsomag, amely pontosan meghatározza az aláírt üzenet típusát, a küldőt és a konkrét adatokat. A beépített domain elválasztó kritikus szerepet játszik a replay attack visszaélések megakadályozásában, ugyanis több adatmezővel és egyedi azonosító mezők alkalmazásával (pl. láncazonosító és szerződéscím megadásával) egyedivé teszi az aláírást, így csak az adott láncon és az adott alkalmazáson belül érvényes az aláírás.

A kész webalkalmazáson fontos szerepet játszanak az aláírások. Lehetővé teszik, hogy minden könyvesemény hozzáadásakor a felhasználó lokálisan aláírjon a 2.1-es ábrán látható módon. Emiatt nem szükséges abban megbízni, hogy a backend nem fogja megváltoztatni az adatokat, mert amint ezt megteszi, a rendszer eltérést jelez, és az aláírás és a tárolt hash eltérése alapján egyértelmű lesz, hogy nem hiteles adat jelenik meg.



2.1 Ábra - Könyv esemény aláírása MetaMaskban

2.4 ERC-1155 token standard

Az ERC-1155 [17] egy Ethereum token standard amely lehetővé teszi egyetlen okosszerződésen belül többféle token típus, akár fungible és non-fungible hatékony használatát. A fejlesztése után, 2018 óta kulcsfontosságú eleme lett a fejlettebb blokklánc alapú játékok, digitális gyűjthető tárgyak és a vagyontárgy tokenizációjának.

Az ERC-1155 kialakításakor a korábbi ERC-20 [18] és ERC-721 [19] standardok problémáinak felülmúlása volt a fő indíték. Ahelyett, hogy minden tokenhez új szerződést kelljen közzétenni, az ERC-1155 lehetővé teszi többféle vagyontárgy reprezentációját, ezzel jelentősen leegyszerűsítve a fejlesztői oldali kezelést.

A gamifikációs funkcionalitás tervezésekor az ERC-1155 mellett az ERC-721 és ERC-20 standard is jelen volt lehetőségként. Az ERC-721 csak egyedi tokenek létrehozását támogatja, ami azt jelentené, hogy minden egyes kitűző egyedi tokenként jelenne meg, ezzel jelentős gas díj és komplexitásnövekedést eredményezve. Az ERC-20 ezzel szemben fungible tokenek kezelésére szolgál, ez ugyancsak nem felelt meg az elvárásoknak. Egyedül az ERC-1155 felhasználásával volt lehetséges megvalósítani egy olyan kitűző mechanikát amely biztosítja minden felhasználó számára, hogy megszerezhesse jutalmát, amely nem egyedi, de egy kitűző fajtából készült saját példánya.

A standard előnyei között említendő a batch műveletek támogatottsága. Több token egyszerre kezelhető, ezzel a tranzakciók száma jelentősen csökken, batch

műveletek révén több token kezelhető egyetlen tranzakcióban, emiatt lényegesen kevesebb gas díj kerül kifizetésre. Rendkívül jól skálázható és egyszerűen használható technológiáról van szó.

A webalkalmazás keretein belül többféle kitűző megszerzése lehetséges, többek között első könyv hozzáadása, vagy első esemény bejegyzés után jár minden felhasználónak egy-egy jutalom amelyeket a profiloldalon tekinthet meg. Ilyenkor egy adott token típus példánya kerül az olvasóhoz, tehát egyfajta kitűzött számos felhasználó birtokolhat. A PostgreSQL adatbázison és a blokkláncon is jelen van minden vagyontárgy, a rendszer lehetőséget biztosít a tulajdonjog és hitelesség ellenőrzésére egy publikus blokklánc böngészőn keresztül (pl. Etherscan) [20].

2.5 QR-kód alapú azonosítás

A QR-kód technológia egy rendkívül hasznos eszköz a fizikai és a digitális felületek összekötéséhez. Egyszerű módon alakít át akármilyen karakterláncot egy kétdimenziós kód formájába ami minden modern telefon kamerájával beolvasható. Lehetővé teszi akár egy plakáton linkeket elhelyezni amit bárki meglátogathat anélkül, hogy be kellene gépelnie egy hosszú hivatkozást a böngészőbe.

A könyvek és a webalkalmazás között is szükség volt egy hasonló megoldásra. Nyomtatott médiumoknál egyértelmű azonosító lehet az ISBN szám [21], amivel beazonosítható a mű, azonban nem alkalmas az egyes példányok egyedi azonosítására, mivel minden kiadáshoz tartozik egy azonosító szám. A QR-kódok egyszerű működtetése miatt választottam használatát.

Hátulütője az ISBN számmal ellenben, egy QR-kód nincs gyárilag az adott köteten, ezért kellett egy híd a fizikai és digitális verzió között, amit egy egyedi grafikus könyvjelzővel valósítottam meg. A könyv felületén elérhető egy könyvjelző generáló gomb. Aktiválásakor elindítja egy képfájl letöltését, melyen szerepel egy rövid tájékoztató, hogy a könyv tagja a könyv nyilvántartó alkalmazásnak. A kép alsó háromnegyedében látható a QR-kód mely az alkalmazás domain címéből és a könyv azonosító karaktersorozatából készül. Amennyiben nem elérhető a felhasználó számára a QR kódok beolvasása, a grafika alján elhelyezésre kerül egy manuálisan megadható azonosító.

A könyvjelzőt kinyomtatni, amely a 2.2-es ábrán látható, majd a nyomtatványt a kötetbe helyezni a könyvet először regisztráló felhasználó feladata. Amennyiben egyszer elhelyezésre került, többször beolvasható, majd továbbadható, új könyvjelzőre nincs szükség. Így egyértelműen azonosítható, hogy mely könyvek szerepelnek a rendszerben.

A QR-kód önmagában nem biztosít hitelességet, azonban a beolvasott adatok blokkláncon történő ellenőrzése garantálja azok integritását.



2.2 Ábra - QR-kóddal ellátott könyvjelző

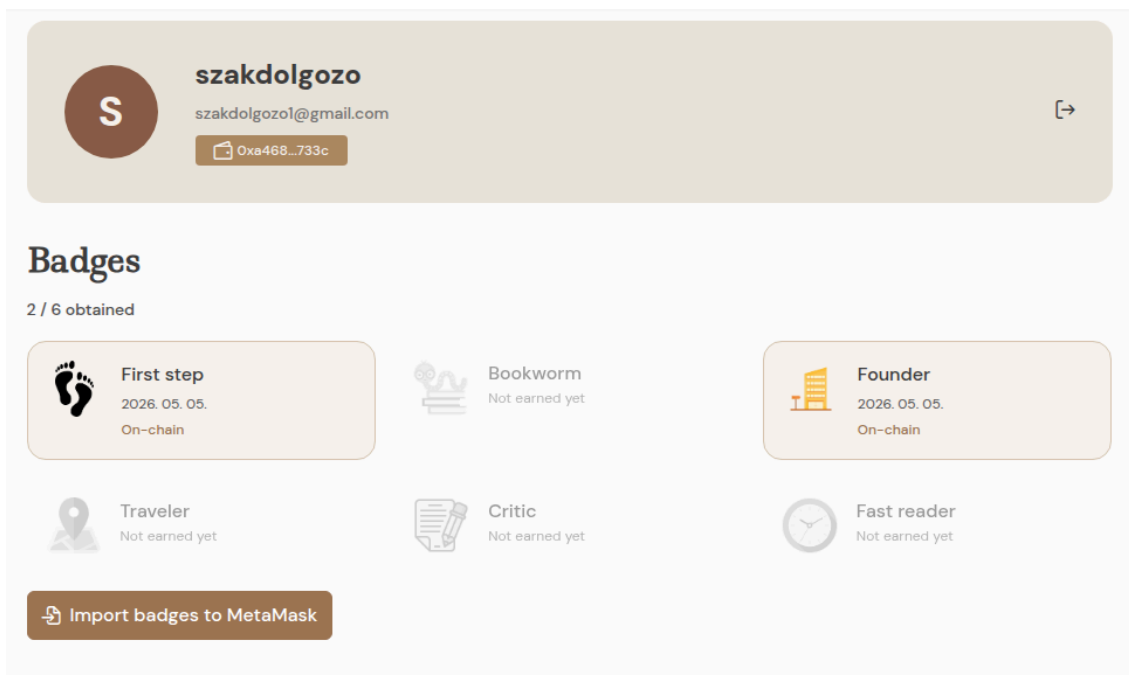
2.6 Gamifikáció alapjai

Napjainkban a gamifikáció széles körben alkalmazott módszer az élet sok területén, különösen a digitális szolgáltatások körében. Ennek az oka az emberi agy jutalmazási rendszere. Amikor egy cselekvés jutalomérzetet nyújt, növeli annak valószínűségét, hogy egy személy többször végzi az említett tevékenységet.

Számos változata lehet a digitális gamifikációnak. Léteznek szintezési rendszerek, tárgygyűjtés, napi bejelentkezésért járó jutalom és még sok más. A rendszerben a 2.3-as ábrán látható kitűzők alkalmazása azért előnyös, mert jól tükrözheti a felhasználó aktivitását és viselkedési mintáit. A különböző kitűzők másfajta teljesítményt követelnek meg. Az aktív, különböző helyszíneken tevékenykedő felhasználók számára elérhető a Traveler kitűző. Eléréséhez legalább öt eltérő

lokációval rendelkező eseményt szükséges közzétenni. Ezek alapján következtetni lehet a felhasználó aktivitására.

Fontos szerepet játszik a felhasználók megtartásában. Ha nem érzékel előrehaladást vagy visszacsatolást, könnyen megfeledkezhet a szolgáltatásról. Érdeemes azonban nem túlzottan kiemelni a gamifikációt, hiszen gyorsan elveszhet a rendszer fő funkciója, ha csak az előrehaladás vezérli az olvasókat. A gamifikáció célja nem a fő funkció kiváltása, hanem annak támogatása és kiegészítése.



2.3 Ábra - Kitűzők a profil oldalon

3. Követelményanalízis

A követelményanalízis a szoftverfejlesztési folyamatok egyik első és talán legfontosabb építőeleme. A fejlesztendő rendszerrel szemben elvárt funkciók feltárása és dokumentálása kritikus szerepet játszik a tényleges tervezési folyamatok megkezdése előtt. A megfelelően meghatározott követelmények hozzájárulnak a fejlesztés strukturáltságához, valamint csökkentik a későbbi módosítások és hibák kockázatát.

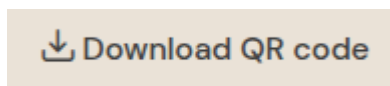
A követelmények meghatározása során fontos különbséget tenni a felhasználói és a fejlesztői szempontok között. Míg a felhasználói nézőpont elsősorban a rendszer funkcionalitására és használhatóságára fókuszál, addig a fejlesztői szempont a megvalósítás technikai részleteit, korlátait és lehetőségeit veszi figyelembe.

A fejezet célja a rendszer működéséhez szükséges alapvető követelmények bemutatása, amelyek funkcionális és nem funkcionális kategóriákba sorolhatók. Ezen felül a legfontosabb felhasználási esetek is ismertetésre kerülnek, amelyek szemléltetik a rendszer használatának tipikus folyamatait.

3.1 Funkcionális követelmények

A funkcionális követelmények meghatározása során a fő szempont a rendszer működése, a problémákat megoldó funkciók viselkedésének dokumentálása. A követelményanalízis ezen pontján az kerül figyelembe, hogy mire képes a rendszer. Figyelembe kell venni, miként fog a felhasználó interakcióba lépni a rendszerrel, ennek egyértelműen és zökkenőmentesen kell történnie.

Felhasználói szempontból vizsgálva, a webalkalmazás magja a könyvek hozzáadása. Minden regisztrált felhasználónak lehetőséget kell biztosítani új könyvek rögzítésére, cím, szerző és borítókép megadását követően. Ahogy az alkalmazás minden funkciójának, a könyv hozzáadásának is elengedhetetlen azonnali visszajelzést adnia az interakció sikerességéről vagy a közbelépő hiba pontos és érthető jellegéről. Sikeres hozzáadást követően, az alkalmazás a 3.1-es ábrán látható gombbal teszi lehetővé a QR-kód generálást, ezzel erősítve a fizikai és a digitális példány kapcsolatát.

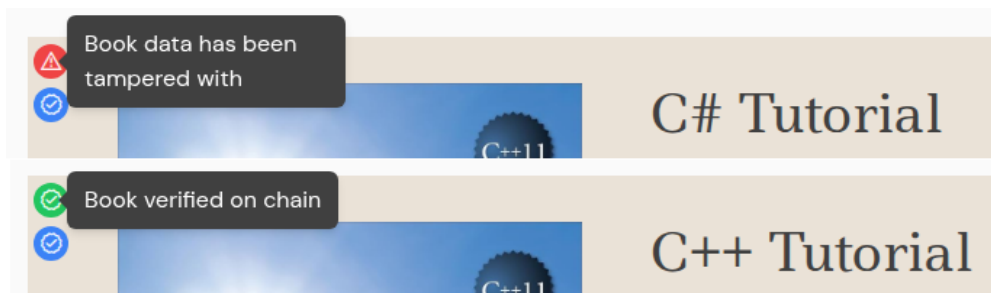


3.1 Ábra - QR-kód letöltőgomb

A második funkció a prioritási listán a könyvesemények rögzítése, megtekintése. Valamennyi regisztrált olvasónak lehetősége van intuitív módon eseményeket hozzáadni, ezzel bővíteni az adott könyv történetét. Szövegbeviteli mezőkkel megadható a helyszín, értékelés és tetszőleges hosszúságú szöveg megadása. Emellett legfeljebb három kép feltöltése engedélyezett. Az események időrendben való tárolása kulcsfontosságú az átláthatóság érdekében. A könyv története a könyvjelzőn található QR-kód vagy hivatkozás segítségével érhető el.

Az alkalmazásnak az adatintegritás és az esetleges módosítások követésére lehetőséget kell biztosítani érthető formában. Bármely könyv vagy könyvesemény megtekintésekor automatikus adatellenőrzés történik. Amennyiben minden hash érték

megegyezik, a 3.2-es ábrán látható zöld színű visszajelző ikon jelenik meg az oldalon, amennyiben eltérést észlel a rendszer, piros színű figyelmeztető jelzést jelenít meg.



3.2 Ábra - Hitelességet jelző ikon

A felhasználói előrehaladás visszajelzéséért kitűző rendszernek kell felelnie, amely bizonyos felhasználói aktivitást követően a megtett interakció jegyében egy adott típusú kitűző példányával jutalmazza az olvasót. A rendszer ezen részén is elkerülhetetlen a blokklánc hitelességének felhasználása, ezért szükséges egy publikus blokklánc böngésző (pl. Etherscan) hivatkozásának megjelenítése minden kitűző alatt amit a profiloldalon képes megtekinteni a felhasználó.

A felhasználói követelmények alapján kerülnek meghatározásra a technikai megoldások. Az adatbázis tervezésekor mérlegelni kell, mely adatok a legfontosabbak, a felesleges tárhelyhasználat elkerülése érdekében. Az alkalmazásban szöveges mezők mellett képek tárolására is van lehetőség, ezt a backend kezeli a multer csomag használatával. Méret és fájl kiterjesztés vizsgálata után az ezen feltételeknek megfelelő képek egyedi nevet kapnak, majd az uploads mappába kerülnek, ezek után külön táblában kerülnek rögzítésre, ezzel biztosítva, hogy több kép is tartozhasson egy eseményhez.

A könyvek és az események hozzáadásakor is először a centralizált adatbázisba rögzülnek az adatok, majd az ezekből készült hash érték kerül a blokkláncra. A könyv hozzáadásáért felelős űrlap egyszerre képes könyv és esemény hozzáadására is, így ekkor először a könyv kerül fel az adatbázisra, majd következik az esemény hozzáadása, ezzel párhuzamban azonos sorrendben történik a blokkláncra rögzítésük is.

Bármely könyv adatainak lekérésekor az aktuális adatbázisban talált bejegyzés hashelésre kerül, majd automatikus összehasonlítás kezdődik a blokkláncsal. Amíg az adat blokkláncon történő rögzítése folyamatban van, sárga színű visszajelzést kap a felhasználó az éppen folyamatban lévő hozzáadásról.

A felhasználói aktivitás folyamatos ellenőrzése szükséges. Amennyiben az olvasó teljesített egy feltételt, a feladathoz kötött kitűzővel kell jutalmazni. Az ilyen esemény aktiválja a kitűző adatbázisban történő rögzítését, majd az okoszerződés token kibocsátási folyamatát. Hasonlóan a könyvekhez és az eseményekhez, egy folyamatban lévő kiírás látható a validálatlan kitűzők mellett, majd egy Etherscan linkké alakul amint sikeresen rögzítésre került a blokkláncon. Mivel a Sepolia testnet [22] sajátos működése miatt a felhasználó nem látja automatikusan a tárcájában az ERC-1155 token, egy külön gomb biztosítja a token megjelenítésének lehetőségét.

A visszaélések elkerülése érdekében kulcsfontosságú minden beviteli mező validációja mind a frontenden, mind a backenden. Fontos a felhasználói élmény erősítéséhez, hogy a mezők kitöltése közben azonnali visszajelzést adjon a megadott adat helyességéről. Kényelmetlen működést eredményez a felhasználó számára, ha csak az űrlap leadásakor szembesül az adatok helytelenségével.

3.2 Nem funkcionális követelmények

A nem funkcionális követelményekben felsorolandó elvárások nem a rendszer működési elveivel kapcsolatosak, hanem annak minőségével. Ebbe beletartozik a sebesség, a biztonság és a használhatóság.

A rendszernek mindenekelőtt gyorsnak kell lennie. A teljesítmény az egyik legfontosabb szempont egy alkalmazás tervezésekor. Sok frusztráció keletkezhet a felhasználóban, ha sokat kell várnia pár utasítás végrehajtására. Minden esetben elengedhetetlen az olvasók folyamatos tájékoztatása a történetekről, akár folyamatban lévő hozzáadás vagy törlésnél, mindig kell lennie egy indikátornak, hogy sikerült elindítani a folyamatot, de még várnia kell az eredményre.

Az API [23] hívásoknak alacsony késleltetéssel kell működnie, viszont minden szükséges validációt meg kell tennie az adatok helyessége megőrzése érdekében. Emiatt fontos az adatbázis szintjétől egészen a frontendig optimalizálásokat végrehajtani.

A blokklánc lekérdezései költségigényesek, emiatt nem tanácsos minden kattintásra a legújabb adatot előállítani. A gyorsítótárazott megoldás régóta alkalmazott az ehhez hasonló rendszereknél. Amennyiben a hozzáadás megtörtént, de még nincs validálva a láncon, fontos jelezni a felhasználó felé, hogy folyamatban van a hitelesítés.

Viszont feleslegesen gyors egy rendszer, ha nem megbízható. A biztonságra kifejezetten sok figyelmet kell áldozni, fontos a folyamatos tesztelése különféle visszaélések ellen. A frontenden és a backenden is egyaránt validálni szükséges minden adatmozgást. Fájlfeltöltésnél érdemes a méretet ellenőrizni a tárhelyoptimalizált működés érdekében, illetve a típusellenőrzés sem maradhat el. A jogosultságkezelés is fontos eleme egy adatokat kezelő alkalmazásnak. Vannak olyan adatok amiket nem változtathat bármely felhasználó.

A blokkláncon tárolt bejegyzések a legfontosabbak, hiszen ehhez viszonyítja a rendszer az adatokat. Amennyiben a blokkláncra helytelen adat kerül, értelmét veszti a funkció. A felhasználói hitelesítést megtestesítő EIP-712 aláírás helyes működése elengedhetetlen. A digitális aláírás hiányában a backend kezeli a blokkláncal való kapcsolatot így megtörve a teljes decentralizáltságot.

Fontos szempont, hogy sok egyidejű felhasználó esetén se csökkenjen a felhasználói élmény. Minél több az egyidejű aktivitás, annál nagyobb az adatbázis és az API terhelése. Emellett a képfeltöltés lehetősége miatt a tárhelyigény is növekszik. Terhelés szempontjából a blokklánc a legfontosabb, ugyanis a láncban való adatmozgás jelentős anyagi ráfordítást igényel. Emiatt fontos, hogy csak a feltöltendő adatok hash értékét tároljuk, ezzel számottevő gas díjat spórolva.

A használhatóság is kulcsfontosságú építőeleme egy modern alkalmazásnak. Az alkalmazásnak intuitívan kell működnie, ugyanis egyre kevesebb időt töltenek kereséssel a felhasználók. Esélyes, hogy ha egy olvasó nem találja meg azonnal amit keres akkor elhagyja az alkalmazást. A QR-kód gyors elérése elengedhetetlen, ez a fő funkció a könyv hozzáadása mellett.

3.3 Felhasználási esetek

A felhasználási esetek, vagy use case-ek olyan leírások, amelyek lépésről lépésre írják le egy adott cél eléréséhez szükséges lépéseket egy felhasználó számára. Dokumentálásuk fontos, mivel jól láthatóvá válik, mely funkciókban található esetlegesen felesleges lépés, vagy túlbonyolítás.

Egy könyv feltöltése a regisztrációval vagy bejelentkezéssel kezdődik. A felhasználó az autentikációt követően megnyitja a feltöltési felületet, majd kitölti az űrlapot a könyv és az első esemény adataival. Feltölti a szükséges képeket és aktiválja a

hozzáadás gombot. Az esemény hozzáadásához szükséges aláírást megerősíti a felhasználó, ezt követően indul a validációs folyamat. A hozzáadást követően azonnali visszajelzést kap az olvasó a hozzáadás eredményességéről. Eredménytelen esetben a rendszer a hiba forrását pontosan kiírja a felhasználó számára egyértelmű formában.

A már feltöltött könyvek a hivatkozás birtokában bármely felhasználó megtekintheti, viszont eseményekkel annak történetét bővíteni csak autentikációt követően lehetséges. Ehhez az oldal alján található gomb aktiválásával megjelenő űrlap kitöltése szükséges. A hozzáadás gomb aktiválása után szintén aláírás szükséges, majd elindul a validációs folyamat.

Amennyiben a felhasználó olyan fizikai könyvhöz jut amelyben megtalálható egy korábban kinyomtatott könyvjelző, a felhasználónak elegendő, hogy beolvassa az azon található QR-kódot és böngészheti a kötet történetét. A még nem regisztrált könyveket hozzá kell adni, majd a példány oldalán a QR-kód letöltésére szolgáló gombra kattintva létrejön a könyvjelző. A grafikát kinyomtatva és a könyvbe helyezve készen áll a továbbadásra más felhasználók számára.

4. Rendszerterv

A fejezet célja a követelményeknek megfelelő megvalósítás bemutatása és dokumentálása. A korábban meghatározott funkcionális és nem funkcionális elvárások alapján kerül kialakításra a rendszer felépítése, amely biztosítja a kívánt működés megvalósítását.

A fejezet során bemutatásra kerül a rendszer architektúrája, beleértve a frontend, backend, adatbázis és blokklánc komponenseket, valamint ezek egymással való kapcsolatát és együttműködését.

A bemutatás magasabb szintű áttekintést ad a teljes rendszerről, amely megalapozza a későbbi részletesebb implementációs és működési leírásokat.

4.1 Architektúra áttekintés

A modern webalkalmazások fejlesztésénél elengedhetetlen egy olyan keretrendszer használata, amely biztosít komponens alapú fejlesztést. A két legelterjedtebb megoldás a frontend fejlesztéshez a React [24] és az Angular [25]. Az Angular mellett szóltak a korábbi tapasztalataim, valamint a kiterjedt dokumentáció és az elérhető csomagok

széles köre. Az újrafelhasználható komponensek, a könnyű HTTP hívások, életciklus-kezelési lehetőségek és a reaktív űrlapok egyszerű implementációja miatt, az Angular keretrendszer jelentős sebességnövekedéssel járult hozzá a fejlesztési folyamathoz.

A backend megvalósításához egyértelmű választás volt a Node.js [26]. A JavaScript alapú környezet, egyszerű kérelmfeldolgozás és a számtalan csomag, különösen a blokklánc-integráció szempontjából hasznos funkcionalitások miatt. A REST API [27] képezi a hidat az adatbázis és a frontend között, emellett koordinálja a webalkalmazás lényegi működését. Minden endpoint hívást input validálás követ, az adatok biztonságos feldolgozása érdekében.

Az off-chain, centralizált adatbázis fejlesztéséhez a PostgreSQL-t vettem igénybe. A relációs adatbázisoknak számos előnye van a nem relációs (NoSQL) adatbázisokkal szemben. A legfőbb előnye a konzisztens, strukturált felépítése a nem relációs adatbázisok JSON vagy BSON alapú kulcs-érték párok rugalmasabb, de kevésbé szigorúan strukturált működésével szemben. Gyors, az SQL programozási nyelvvel megírandó lekérdezések végrehajtását teszi lehetővé, ami fontos szempont a teljesítmény optimalizáltságához.

A blokklánc funkciók fejlesztéséhez az egyik legszélesebb körben használt fejlesztőkörnyezetet, a Hardhat-et [28] vettem igénybe. Az Angular és a Node.js Typescript és JavaScript jellegű szintaxisa mellé tökéletesen beillik a JavaScript alapú Hardhat is. Számos rendkívül hasznos eszközt tartalmaz, mint például lokális blokklánc hálózat vagy a beépített tesztelés. A Solidity nyelven íródott okosszerződés biztosítja az adatok integritását, hitelességét, és a tranzakciók pontos végrehajtását. A kimagasló költségeket és a teljesítményromlás elkerülése érdekében minden tárolni kívánt adat hash értéke kerül a láncra.

4.2 Adatbázis tervezés

A rendszer fejlesztését mindenképp relációs adatbázissal terveztem megvalósítani. A strukturált tárolás jelentősen megkönnyíti az adatok átlátható kezelését.

A tervezést az autentikációval kezdtem, ehhez szükség volt két adatbázis táblára. A users tábla tartalmazza a felhasználók alapvető adatait, mint például az email cím, felhasználónév, a jelszó hash értéke. Ezek mellett az elsődleges kulcsként szolgáló id és

a fiók létrejöttének időbélyege is helyet kapott. Az access és refresh tokenekkel megvalósított munkamenet fenntartásához szükség van egy táblára ami a refresh tokeneket tárolja. A legfőbb mezői közé tartozik a users.id ami egy reláció, a felhasználó azonosítója, maga a token, emellett tárolásra kerül egy egyedi azonosító, a létrehozás és lejárat időbélyege is.

A felhasználói fiókok után a könyvek modellje következett. A kötelező cím, szerző mezőjén kívül szükséges tárolni a borítókép elérési útvonalát, a feltöltő felhasználójához egy referenciát, a primer kulcsot és az időbélyeget. A könyvek és a felhasználói fiókok közötti kapcsolat úgynevezett one-to-many, tehát egy felhasználó tetszőleges számú könyvet adhat hozzá, viszont a könyv eredeti feltöltője egy felhasználóhoz köthető.

Az események kapcsolata a felhasználóval hasonló a könyvekhez. Azonos módon egy esemény egy felhasználóhoz tartozik, de akármennyit tud létrehozni. A különbség a könyvekkel való kapcsolat. Minden esemény kapcsolódik pontosan egy könyvhöz, de egy könyvhöz bármennyi bejegyzés kapcsolódhat. A referenciákon kívül feljegyzésre kerül a lokáció, ahol az olvasó időt töltött a könyvvel, egy leírás, egy 1 és 5 közötti értékelés és ezen adatok hash értéke. Hasonlóan az eddigi táblákhoz, az eseményekhez is tartozik azonosító és időbélyeg.

A könyvek és az események között fontos különbség, hogy a bejegyzések esetében külön hash érték kerül tárolásra, míg a könyvek esetében ez a mező jelenleg nem szerepel az adatmodellben. Ennek oka, hogy a rendszer tervezése során az események hitelesítése kapott elsődleges szerepet, mivel ezek reprezentálják a könyvek életútjának dinamikusan változó elemeit. A könyvek blokkláncon történő rögzítése a fejlesztési folyamat későbbi szakaszában merült fel mint kiterjesztési lehetőség. Ennek megfelelően a jelenlegi modell úgy lett kialakítva, hogy az később egyszerűen bővíthető legyen, például külön hash mező vagy dedikált hash-tábla bevezetésével. Ez a megközelítés lehetővé teszi a rendszer továbbfejlesztését anélkül, hogy a meglévő adatstruktúra jelentős átalakítására lenne szükség.

Az eseményekhez kapcsolódik egy kiegészítő tábla, amely a hozzájuk kapcsolódó képek elérési útvonalait tárolja. Erre a több képfeltöltési lehetőség miatt volt szükség. Az azonosító mellett egy referencia az eseményhez és a kép URL alkotja a táblát.

A kitűzők tárolásához szintén két táblára van szükség. Az egyik maga a kitűző sémáját tartalmazza, a másik a felhasználói birtoklást biztosítja. Egy kitűző primer kulcsból, kép elérésből és névből áll. A kapcsolatot biztosító tábla egy kapcsolótáblaként szolgál a kitűző és a felhasználó között. Egyedi azonosító mellett a felhasználó referenciáját, a kitűző azonosítóját és a megszerzés időbélyegét tartalmazza.

Az alkalmazás egy mellékfunkciója biztosít egyfajta idővonalat ahol megjelennek a legfrissebb aktivitások. Az egész adatbázison átívelő metaadatok mellett az aktivitást előidéző felhasználó, a cselekvés típusa, mint például könyv hozzáadás, vagy új felhasználói regisztráció kerül rögzítésre. Emellett az entitás is kapott saját mezőt, ezzel biztosítva a hivatkozásokat az adott aktivitás és az érintett entitás között.

4.3 Blockchain architektúra

A webalkalmazás fejlesztése során hibrid adattárolási megoldás került alkalmazásra. Míg a blokkláncon kizárólag a könyvekhez, eseményekhez és kitűzőkhöz tartozó hash értékek kerülnek rögzítésre, addig a centralizált adatbázisban a teljes metaadatok, képek és felhasználói adatok találhatóak meg. A blokklánc ebben az architektúrában nem elsődleges adattárolási réteggént, hanem hitelesítési mechanizmusként funkcionál. Az adatbázis gyors és hatékony elérést biztosít, míg a blokklánc az adatok integritását garantálja.

Az Ethereum működéséből adódóan minden tárolt adat költséggel jár, ezért fontos a láncban tárolt adatmennyiség minimalizálása. Ennek megfelelően kizárólag hash értékek kerülnek rögzítésre, amely jelentősen csökkenti a tranzakciókhoz kapcsolódó gas költségeket. A hash alkalmazása további előnnyel is jár. Fix méretéből adódóan kiszámítható költséget biztosít, valamint egyirányú jellege miatt nem teszi lehetővé az eredeti adatok visszaállítását, így érzékeny információk sem kerülnek nyilvános blokkláncra. Emellett a hash értékek alkalmasak az adatok változatlanóságának bizonyítására.

Az adatfeldolgozás folyamata során az adatok elsőként a centralizált adatbázisba kerülnek rögzítésre, majd ezekből hash érték kerül generálásra. A generált hash ezt követően kerül rögzítésre a blokkláncon. Lekérdezés során a blokkláncon tárolt hash és az aktuális adatbázis állapotából újraszámolt hash kerül összehasonlításra. A két érték

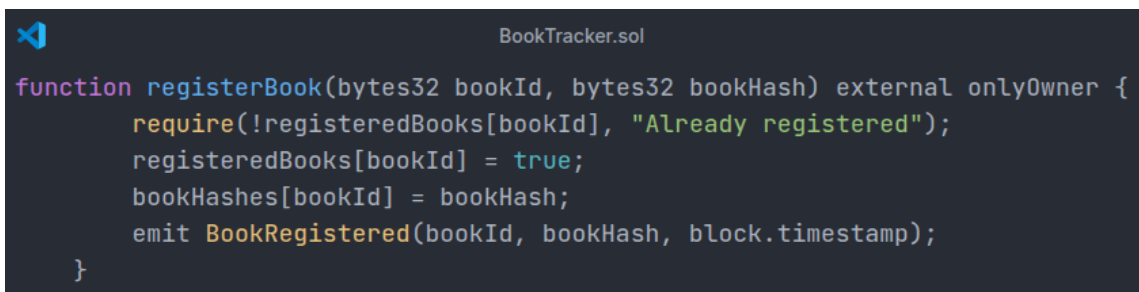
egyezése igazolja az adatok hitelességét, míg eltérés esetén megállapítható az adatok módosulása.

4.4 Okoszerződés tervezése

A webalkalmazás blokklánc alapú funkcionalitásának megvalósításához egy Solidity nyelven íródott okoszerződés került kialakításra, amely a rendszer hitelesítési rétegeként funkcionál. Feladata a könyvekhez és eseményekhez tartozó hash értékek rögzítése, valamint a gamifikációs rendszerhez kapcsolódó kitűzők kezelésének biztosítása. Az okoszerződés működése garantálja, hogy a láncra felkerülő adatok utólag nem módosíthatók, ezzel biztosítva a rendszer integritását.

A szerződés logikája úgy lett kialakítva, hogy minimalizálja a láncon tárolt adatmennyiséget, ennek megfelelően kizárólag hash értékek kerülnek rögzítésre. A könyvek és események azonosítása egyedi azonosítók segítségével történik, amelyekhez a hozzájuk tartozó hash értékek kerülnek társításra. Az adatok tárolása mapping struktúrák segítségével valósul meg, amely gyors és hatékony elérést biztosít a szerződésen belül. A műveletek során minden esetben ellenőrzésre kerül, hogy az adott entitás már létezik-e, ezzel elkerülve a duplikációt és a hibás állapotok kialakulását.

A könyvek blokkláncon történő regisztrálásáért a 4.1-es ábrán bemutatott registerBook függvény felel. A függvény bemenetként egy egyedi azonosítót és a könyvhöz tartozó hash értéket vár, majd eltárolja azt a szerződés állapotában. A művelet során ellenőrzésre kerül, hogy az adott könyv már szerepel-e a rendszerben, így biztosítva, hogy egy könyv csak egyszer kerülhessen rögzítésre. A sikeres végrehajtást követően egy esemény kibocsátására is sor kerül, amely lehetővé teszi a tranzakciók nyomon követését.



```
BookTracker.sol

function registerBook(bytes32 bookId, bytes32 bookHash) external onlyOwner {
    require(!registeredBooks[bookId], "Already registered");
    registeredBooks[bookId] = true;
    bookHashes[bookId] = bookHash;
    emit BookRegistered(bookId, bookHash, block.timestamp);
}
```

4.1 Ábra - registerBook függvény

A könyvekhez kapcsolódó események rögzítését a 4.2-es ábrán látható logEvent függvény biztosítja. A működése során egy meglévő könyvhöz kapcsolódó új esemény

hash értéke kerül eltárolásra. A függvény ellenőrzi a könyv létezését, ezzel biztosítva a konzisztens adatkapcsolatot. Az események egymás után történő rögzítése egyfajta láncolt történetet hoz létre, amely a könyv teljes életútját reprezentálja.

```
BookTracker.sol

function logEvent(
    bytes32 bookId,
    bytes32 eventHash,
    address signer,
    bytes calldata signature
) external onlyOwner {
    require(registeredBooks[bookId], "Book not registered");
    require(!loggedEvents[eventHash], "Event already logged");

    if (signer != address(0)) {
        bytes32 digest = _hashTypedDataV4(keccak256(abi.encode(
            EVENT_TYPEHASH,
            bookId,
            eventHash,
            signer,
            nonces[signer]
        )));

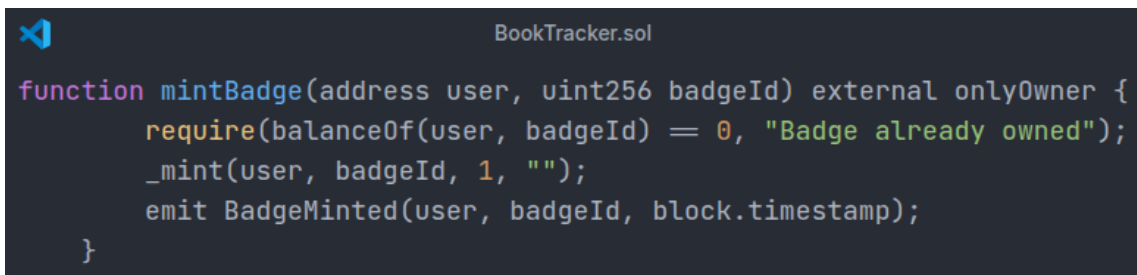
        address recovered = ECDSA.recover(digest, signature);
        require(recovered == signer, "Invalid signature");

        nonces[signer]++;
    }

    loggedEvents[eventHash] = true;
    emit EventLogged(bookId, eventHash, signer, block.timestamp);
}
```

4.2 Ábra - logEvent függvény

A gamifikációs rendszer blokklánc oldali megvalósításáért a 4.3-as ábrán található mintBadge függvény felel, amely az ERC-1155 token szabványra épül. A függvény lehetővé teszi különböző típusú kitűzők létrehozását és kiosztását a felhasználók számára. A tokenek egyedi azonosítóval rendelkeznek, azonban egy adott típusból több példány is létezhet, amely megfelel a kitűző alapú jutalmazási rendszer működésének. A kitűzők kiosztása meghatározott feltételek teljesülése esetén történik, amelyeket a backend rendszer vizsgál, majd a szerződés megfelelő függvényének meghívásával történik a tényleges mintelés.



```

BookTracker.sol

function mintBadge(address user, uint256 badgeId) external onlyOwner {
    require(balanceOf(user, badgeId) == 0, "Badge already owned");
    _mint(user, badgeId, 1, "");
    emit BadgeMinted(user, badgeId, block.timestamp);
}

```

4.3 Ábra - *mintBadge* függvény

A szerződés működése során kiemelt szerepet kap a biztonság és az adatok helyességének ellenőrzése. Minden állapotmódosító művelet feltételekhez kötött, amely biztosítja, hogy csak érvényes és konzisztens adatok kerüljenek rögzítésre. A tranzakciók során alkalmazott digitális aláírások garantálják, hogy a műveleteket valóban az arra jogosult felhasználók kezdeményezik.

A költséghatékony működés érdekében a szerződés minimalizálja a tárolt adatok mennyiségét és egyszerű adattípusokat alkalmaz. A teljes metaadatok láncon történő tárolása jelentős gas költséggel járna, ezért a rendszer kizárólag a hash értékek rögzítésére épül. Ez a megközelítés biztosítja az adatok hitelességét, miközben optimalizálja a tranzakciók költségét.

Az okosszerződés szorosan együttműködik a backend rendszerrel, amely előállítja a szükséges hash értékeket, valamint kezeli a tranzakciók indítását. A szerződés ezen adatok validálásáért és rögzítéséért felel, így egyértelműen elkülönül a rendszer logikai és bizalmi rétege.

4.5 Backend működése

A webalkalmazás backend része egy Node.js alapú REST API, amely a frontend, az adatbázis és a blokklánc közötti kommunikációt biztosítja. Feladata a beérkező kérések feldolgozása, az adatok validálása, valamint a szükséges műveletek végrehajtása. Az adatok minden esetben először a centralizált adatbázisba kerülnek, majd ezt követően történik meg a blokkláncra történő rögzítés.

A blokklánccal való kapcsolat kezeléséért egy külön komponens, a BlockchainService felel. Ez a réteg végzi a hash értékek előállítását és továbbítását az okosszerződés felé, valamint a tranzakciók indítását és azok visszaigazolásának kezelését. A könyvek esetében a rendszer egyedi azonosítóból képzett hash segítségével hoz létre egy láncon tárolt reprezentációt, míg az eseményeknél a backend által generált

hash kerül rögzítésre. A tranzakciók sikeres végrehajtását követően a visszakapott tranzakció hash kerül eltárolásra, amely később hivatkozási alapként szolgál.

A tranzakciók helyes sorrendben történő feldolgozásához elengedhetetlen a nonce értékek kezelése. Mivel egy adott tárcához tartozó tranzakciók csak növekvő nonce értékkel fogadhatók el, a backend minden tranzakció indítása előtt lekérdezi az aktuális nonce értéket, beleértve a még függőben lévő tranzakciókat is. Ez biztosítja, hogy párhuzamos műveletek esetén se alakuljon ki ütközés vagy sikertelen tranzakció.

A backend további feladata a blokklánc állapotának figyelése is. Egyes műveletek esetében szükséges megvárni, amíg egy adott tranzakció véglegesen rögzítésre kerül a láncon. Ennek érdekében, a 4.4-es ábrán megfigyelhető módon, a rendszer lekérdezésekkel ellenőrzi például egy könyv regisztrációjának állapotát, és csak ezt követően teszi elérhetővé a további műveleteket a felhasználó számára.

A backend ezen felül központi koordinációs szerepet tölt be a rendszerben, mivel összekapcsolja a gyors elérésű adatbázist a megbízható blokklánc réteggel, biztosítva az adatok konzisztens és hiteles kezelését.

A screenshot of a code editor with a dark background. The file name 'blockchain.service.js' is visible at the top. The code is written in JavaScript and defines an asynchronous function 'waitForBookRegistration'. It takes 'bookUuid' and 'maxRetries' (default 30) as arguments. Inside the function, it converts the UUID to a keccak256 hash. A 'for' loop iterates from 0 to 'maxRetries'. In each iteration, it calls 'contract.isBookRegistered' with the hash. If it returns true, the function returns true. Otherwise, it logs a message and waits for 2000ms using 'setTimeout' before the next attempt. If the loop completes without success, it returns false.

```
blockchain.service.js

export async function waitForBookRegistration(bookUuid, maxRetries = 30) {
  const bookId = ethers.keccak256(ethers.toUtf8Bytes(bookUuid));

  for (let i = 0; i < maxRetries; i++) {
    const isRegistered = await contract.isBookRegistered(bookId);
    if (isRegistered) return true;
    console.log(
      `Waiting for book registration... attempt ${i + 1}/${maxRetries}`,
    );
    await new Promise((resolve) => setTimeout(resolve, 2000));
  }
  return false;
}
```

4.4 Ábra - *waitForBookRegistration* függvény

4.6 Frontend működése

A frontend megvalósítása az Angular keretrendszerrel történt, amely strukturált, moduláris felépítést tesz lehetővé. Az alkalmazás komponens alapú architektúrát követ, ahol a felhasználói felület kisebb, újrafelhasználható egységekre van bontva. Ilyenek például a könyv és esemény kártyák, űrlap komponensek, valamint a navigációs és

dialógus elemek. Ez a megközelítés elősegíti a kód átláthatóságát és a későbbi bővíthetőséget.

Az Angular routing rendszer biztosítja az egyes oldalak közötti navigációt, például a könyv részletező, profil vagy felfedező nézetek között. Az alkalmazás szolgáltatás alapú adatkezelést használ, ahol az egyes service-ek felelősek az API kommunikációért, az állapot kezeléséért és az üzleti logika egy részéért. HTTP interceptor gondoskodik az autentikációs tokenek automatikus csatolásáról, míg a guard-ok védik azokat az útvonalakat, amelyek csak bejelentkezett felhasználók számára érhetőek el.

A Web3Service kezeli a blokklánccal kapcsolatos frontend oldali műveleteket. Feladata a wallet kapcsolat létrehozása, a felhasználó címének tárolása, valamint az aláírási folyamat lebonyolítása. Az EIP-712 alapú aláírás során itt történik a domain, típus és adatstruktúra összeállítása, beleértve a nonce kezelését is, amely megakadályozza az aláírások újrafelhasználását. Az aláírás minden esetben a felhasználó tárcájában történik, így a frontend biztosítja, hogy az adatok hitelesítése lokálisan valósuljon meg.

A VerificationService a blokkláncon tárolt és a lokális adatok összevetéséért felel. A szolgáltatás lekérdezi az okosszerződésből a megfelelő hash értékeket, majd összehasonlítja azokat a frontend által számolt értékekkel. Az eredmény alapján többféle állapotot különböztet meg, mint például hiteles, módosított vagy még nem validált adat. Ez az információ közvetlenül megjelenik a felhasználói felületen, így az integritás ellenőrzése egyértelmű és azonnal értelmezhető formában történik.

A felhasználói élmény javítása érdekében az alkalmazás reaktív megközelítést alkalmaz. Az Angular signal alapú állapotkezelése lehetővé teszi, hogy a wallet kapcsolat vagy autentikációs állapot változásai automatikusan frissítsék a felületet. Az űrlapok, a 4.5-ös ábrán megfigyelhető reaktív formokkal készültek, amelyek valós idejű validációt biztosítanak, így a felhasználó már adatbevitel közben visszajelzést kap az esetleges hibákról.

Sign up

Email

w

Invalid email

Username

dw

Username must be at least 3 characters long

Password

Weak

Password must be at least 8 characters long

Password Confirm

Already have an account? [Login](#)

Cancel Sign up

4.5 Ábra - Regisztrációs reaktív űrlap

5. Fejlesztés során felmerülő problémák és megoldások

A fejlesztési folyamat során számos olyan probléma merült fel, amelyekre nem volt elegendő egy egyszerű technikai megoldás alkalmazása. A webalkalmazás több különálló rendszer együttműködésére épül, emiatt a frontend, backend és blokklánc réteg közötti kommunikáció sok esetben extra komplexitást eredményezett. Külön figyelmet kellett fordítani az aszinkron folyamatok kezelésére, az adatok hitelességének megőrzésére és a felhasználói élmény fenntartására. A fejezet bemutatja a fejlesztés során felmerülő fontosabb problémákat és az ezekre alkalmazott megoldásokat.

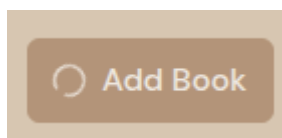
5.1 Blokklánc tranzakciók szinkronizációja

A blokklánc alapú működés egyik legnagyobb nehézsége az aszinkron tranzakciókezelés volt. Míg egy hagyományos adatbázis művelet szinte azonnal végrehajtódik, a blokkláncon minden tranzakciónak idő szükséges a blokkba kerüléshez és validáláshoz. Emiatt külön figyelmet kellett fordítani arra, hogy a rendszer ne próbáljon még nem létező vagy feldolgozás alatt álló adatokat ellenőrizni.

A problémát tovább növelte, hogy a backend egyszerre több tranzakciót is képes indítani rövid időn belül. Az Ethereum hálózat minden tranzakcióhoz egy egyedi nonce értéket használ, amely meghatározza a tranzakció sorrendjét. Amennyiben két tranzakció azonos nonce értékkel kerül elküldésre, az egyik sikertelen lesz. Emiatt minden blokklánc művelet előtt szükséges lekérni az aktuális nonce értéket a pending tranzakciók figyelembevételével.

A tranzakciók feldolgozásának ideje miatt szükség volt várakoztatási mechanizmus bevezetésére is. A `tx.wait()` használata biztosítja, hogy a rendszer csak akkor folytassa a következő műveletet, amikor a tranzakció már ténylegesen blokkba került. Enélkül gyakran előfordult, hogy a frontend már megpróbálta ellenőrizni a hash értéket, miközben az még nem volt elérhető a láncon.

Felhasználói szempontból fontos volt, hogy a rendszer folyamatos visszajelzést adjon a folyamat állapotáról. A blokklánc késleltetése miatt több másodperc is eltelhet egy tranzakció véglegesítéséig, emiatt pending állapot és az 5.1-es ábrán látható betöltési visszajelzések kerültek bevezetésre a felhasználói felületen. Ez jelentősen csökkenti annak érzetét, hogy az alkalmazás lefagyott vagy hibásan működik.



5.1 Ábra - Betöltési visszajelzés gombra helyezve

5.2 EIP-712 aláírás implementációja

Az EIP-712 alapú aláírás implementálása a fejlesztés egyik legérzékenyebb része volt. A rendszer működéséhez elengedhetetlen, hogy a frontend, a backend és az okoszerződés pontosan ugyanazt az adatstruktúrát használja az aláírás előállításakor és ellenőrzésekor. Már egyetlen eltérő mezőnév, adattípus vagy sorrend is teljesen érvénytelen aláírást eredményez.

Az aláírás typed data struktúrával történik, amely tartalmazza a könyv azonosítóját, az esemény hash értékét, a felhasználó wallet címét és a nonce értéket. A frontend oldalon ezekből készül az aláírandó objektum, majd a MetaMask [29] ezt írja alá a felhasználó privát kulcsával. Az okoszerződés ezután ugyanebből az adatszerkezetből számolja újra az aláírás ellenőrzéséhez szükséges hash értéket.

A működés során különösen fontos szerepet kap a frontend és backend közötti hash egyezés. Az esemény hash értékét minden esetben pontosan azonos módon szükséges előállítani. A fejlesztés során több hibát is okozott, hogy eltérő karakterlánc vagy formázás került felhasználásra a különböző rétegekben, emiatt az aláírás ellenőrzése sikertelenné vált annak ellenére, hogy a felhasználó helyesen írta alá az adatokat.

A domain separator szintén kritikus eleme az implementációnak. Tartalmazza az alkalmazás nevét, verzióját, a chainId értéket és az okoszerződés címét. Ez biztosítja, hogy az aláírás kizárólag az adott alkalmazásban és az adott hálózaton legyen felhasználható. Hibás domain adatok esetén az aláírás technikailag érvényes lehet, viszont az okoszerződés nem tudja hitelesként kezelni.

Az aláírás ellenőrzése során további problémát jelentett a replay attack elleni védelem kialakítása. Emiatt minden felhasználóhoz külön nonce érték tartozik az okoszerződésen belül, amely minden sikeres aláírás után növekszik. Ez biztosítja, hogy ugyanaz az aláírás ne legyen újra felhasználható egy későbbi tranzakció során.

5.3 Fájlfeltöltés és médiakezelés

A könyv eseményekhez tartozó képek kezelése külön figyelmet igényelt a fejlesztés során. A rendszernek egyszerre kellett biztosítania a gyors feltöltést, a biztonságos működést és a több kép kezelésének lehetőségét. Az eseményekhez legfeljebb három kép csatolható, melyek az eseményhez kapcsolódó élmények, helyszínek vagy maga a könyv állapotának megjelenítésére szolgálnak.

A backend oldalon a képfeltöltések kezelését a multer csomag végzi. A middleware feladata a multipart form-data típusú kérések feldolgozása, majd a megfelelő fájlok mentése az uploads mappába. A feltöltött képek minden esetben egyedi fájlnevet kapnak, ezzel elkerülhető a névütközés és a korábbi képek felülírása.

A visszaélések és hibák elkerülése érdekében minden feltöltött fájl validálásra kerül. Vizsgálatra kerül a fájl mérete és kiterjesztése is, így csak a támogatott képformátumok kerülhetnek tárolásra. Ez nemcsak biztonsági szempontból fontos, hanem tárhelyoptimalizálás miatt is, ugyanis a képek jelentik a rendszer legnagyobb tárhelyigényű elemeit. A frontend szintén végez előzetes ellenőrzést, így a felhasználó azonnali visszajelzést kap hibás fájl kiválasztásakor.

Az adatbázisban a képek külön event_images táblában kerülnek tárolásra. Ennek oka, hogy egy eseményhez több kép is tartozhat, emiatt szükség volt egy one-to-many kapcsolat kialakítására az események és a médiák között. A táblában kizárólag az esemény referenciája és a kép elérési útvonala kerül eltárolásra, maga a fájl nem az adatbázisban helyezkedik el. Ez jelentősen csökkenti az adatbázis terhelését és egyszerűbbé teszi a médiafájlok kezelését.

5.4 Frontend és wallet integráció

A web3 funkcionalitás egyik legfontosabb eleme a felhasználói wallet kezelése. A decentralizált hitelesítés miatt szükség volt egy olyan megoldásra, amely képes kommunikálni a böngészőben futó Ethereum tárcával. A frontend oldalon ezt a szerepet a MetaMask tölti be, amely jelenleg az egyik legelterjedtebb web3 wallet megoldás.

A wallet kapcsolódás logikája a Web3Service szolgáltatásban került megvalósításra. A kapcsolat létrehozásakor a frontend ellenőrzi, hogy elérhető-e a window.ethereum objektum, majd ethers.js segítségével létrehozza a BrowserProvider példányt. Sikeres csatlakozás után lekérésre kerül a felhasználó címe, amely eltárolásra kerül az alkalmazás állapotában és továbbításra kerül a backend számára is.

Fontos szempont volt, hogy az alkalmazás újratöltésekor ne legyen szükség minden alkalommal manuális újracsatlakozásra. Emiatt készült egy automatikus reconnect logika, amely az alkalmazás indulásakor ellenőrzi a korábban engedélyezett wallet kapcsolatot. Amennyiben a felhasználó nem választotta le manuálisan a tárcát, a rendszer automatikusan visszaállítja a kapcsolatot. Ez jelentősen javítja a felhasználói élményt, hiszen a wallet folyamatos újracsatlakoztatása rendkívül kényelmetlen működést eredményezne.

A MetaMask accountChanged eseményének kezelése szintén elengedhetetlen volt. A felhasználó bármikor válthat aktív wallet címet a böngészőben, ezért a frontend folyamatosan figyeli ezt az eseményt. Amennyiben a tárca címe megváltozik, az alkalmazás automatikusan frissíti a lokális állapotot. Ha a felhasználó teljesen leválasztja a walletet, a rendszer kijelentkezteti a web3 kapcsolatból.

Az Angular frontendben az állapotkezeléshez signal alapú megoldást alkalmaztam. A walletAddress és isConnected állapotok signal formában kerülnek tárolásra, így az ezekre épülő komponensek automatikusan újrenderelődnek változás

esetén. Ez lényegesen egyszerűbb és átláthatóbb működést biztosított, mint a korábban gyakran használt Observable alapú globális állapotkezelés. A wallet kapcsolat állapota emiatt az alkalmazás teljes felületén valós időben frissül.

5.5 Könyv és esemény egyidejű létrehozása

A webalkalmazás egyik legösszetettebb része a könyv és az első esemény egyidejű létrehozása volt. A felhasználói élmény szempontjából fontosnak tartottam, hogy ne külön oldalakon történjen a könyv és az első bejegyzés hozzáadása, hanem egyetlen nagyobb űrlap segítségével. Ez jóval kényelmesebb működést biztosít, viszont fejlesztői oldalról jelentősen bonyolultabb adatkezelést eredményezett.

Az űrlap egyszerre kezeli a könyv metaadatait és az első eseményhez tartozó adatokat. A frontend reaktív formokon keresztül gyűjti össze a szükséges mezőket, majd egyetlen kérésfolyamat indul el. A műveletek sorrendje azonban kritikus szerepet játszik. Elsőként a könyv kerül létrehozásra az adatbázisban, hiszen az esemény csak egy már létező könyvhöz kapcsolódhat. Sikeres könyv hozzáadás után indul az esemény létrehozása.

A blokklánc műveletek szintén egymásra épülnek. A könyv hash értéke először regisztrálásra kerül a láncon, majd ezt követi az esemény hash rögzítése és az EIP-712 aláírás ellenőrzése. A folyamat során több tranzakció is pending állapotba kerülhet, emiatt fontos volt a megfelelő szinkronizáció és a tx.wait használata. Enélkül előfordulhatott volna, hogy az esemény hamarabb próbál a láncra kerülni, mint maga a könyv.

A hibakezelés különösen fontos szerepet kapott ennél a funkcionalitásnál. Több egymásra épülő folyamat miatt számos ponton megszakadhat a művelet. Előfordulhat hibás fájlfeltöltés, sikertelen wallet aláírás, blokklánc tranzakció hiba vagy akár hálózati késleltetés is. Emiatt minden lépés külön validációt és visszajelzést kapott. A frontend toast üzenetekkel és vizuális hibajelzésekkel tájékoztatja a felhasználót a folyamat aktuális állapotáról.

A loading állapotok kezelése szintén elengedhetetlen volt. A blokklánc tranzakciók természetükből adódóan lassabbak, mint egy hagyományos adatbázis művelet, emiatt a felhasználó több másodpercig is várakozhat. A rendszer ezért folyamatos vizuális visszajelzést biztosít a folyamatban lévő validációról és láncra

helyezésről. Ez jelentősen csökkenti annak esélyét, hogy a felhasználó többször próbálja elküldeni ugyanazt az adatot.

6. Tesztelés és kiértékelés

A fejlesztés során kiemelten fontos szerepet kapott a rendszer teljesítményének és stabilitásának vizsgálata. A tesztelés célja nem kizárólag a hibák felderítése volt, hanem annak elemzése is, hogy a webalkalmazás hogyan viselkedik nagyobb terhelés, blokklánc késleltetés és párhuzamos felhasználói aktivitás mellett. Egy decentralizált rendszer esetében különösen fontos figyelembe venni, hogy a hagyományos webes késleltetések mellett a blokklánc saját működéséből adódó várakozási idő is jelentős hatással van a felhasználói élményre.

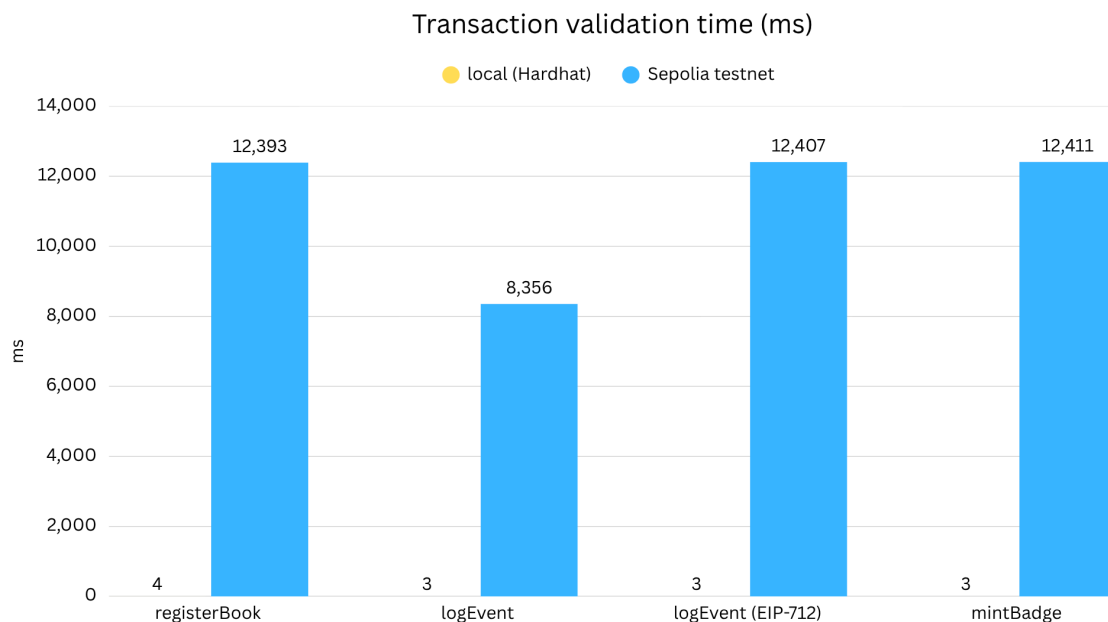
A tesztelés során backend terhelésvizsgálat, blokklánc tranzakció elemzés és válaszidő mérések készültek. A kapott eredmények segítségével jól megfigyelhetővé vált a centralizált és decentralizált komponensek közötti teljesítménykülönbség, illetve az is, hogy a háttérben aszinkron módon végzett blokklánc műveletek milyen mértékben képesek csökkenteni a felhasználó által érzékelt várakozási időt.

6.1 Tesztkörnyezet

A rendszer tesztelése több különböző környezetben történt a lehető legpontosabb eredmények elérése érdekében. A blokklánc funkcionalitások fejlesztéséhez és lokális vizsgálatához a Hardhat fejlesztői környezet került felhasználásra. A beépített lokális Ethereum hálózat lehetővé tette az okosszerződések gyors futtatását, tranzakciók szimulációját és a gas költségek pontos elemzését. Emellett rendkívül hasznosnak bizonyult a nonce kezelés és az EIP-712 aláírások hibakeresése során is.

A lokális hálózat mellett szükség volt egy publikus tesztkörnyezet használatára is. Erre a célra az Ethereum Sepolia testnet került kiválasztásra. A Sepolia működése közel azonos a valódi Ethereum hálózattal, viszont a tranzakciókhoz használt tokeneknek nincs valódi pénzületi értéke. Ez lehetővé teszi a valós környezethez hasonló tesztelést jelentős anyagi ráfordítás nélkül. A rendszer wallet integrációja, az EIP-712 aláírások és az ERC-1155 tokenek működése is ezen a hálózaton került végleges ellenőrzésre.

A két környezet közötti különbség leginkább a tranzakciók megerősítési idejénél volt megfigyelhető. Ahogy a 6.1-es ábrán megfigyelhető, míg a Hardhat lokális hálózatán egy tranzakció néhány milliszekundum alatt véglegesedik, addig a Sepolia hálózaton ez több másodpercet is igénybe vehet. Emiatt a fejlesztés során külön figyelmet kellett fordítani az aszinkron működésre és a loading állapotok megfelelő kezelésére.



6.1 Ábra - Hardhat local és Sepolia testnet validációs idők

6.2 Funkcionális tesztek

A funkcionális tesztek célja annak ellenőrzése volt, hogy az okosszerződés és a hozzá kapcsolódó backend logika megfelelően működik-e különböző helyzetekben. A tesztelés Hardhat környezetben történt automatikus unit tesztek segítségével. A vizsgálatok során nemcsak a sikeres működés került ellenőrzésre, hanem a hibás vagy rosszindulatú használati esetek is.

A könyv létrehozás tesztjei ellenőrizték a registerBook függvény működését. Vizsgálatra került a sikeres regisztráció, a megfelelő hash eltárolása és a BookRegistered esemény kibocsátása. Emellett külön teszt készült annak ellenőrzésére, hogy ugyanaz a könyv ne kerülhessen többször regisztrálásra. A jogosultságkezelés is tesztelve lett, így csak a szerződés tulajdonosa képes új könyvet láncra helyezni.

Az események naplózásánál a logEvent működése került ellenőrzésre. A rendszer sikeresen képes eseményt rögzíteni regisztrált könyvhöz, valamint

megakadályozza esemény létrehozását nem létező könyvek esetében. Vizsgálatra került a duplikált események blokkolása is, így ugyanaz az esemény hash nem kerülhet többször a láncra.

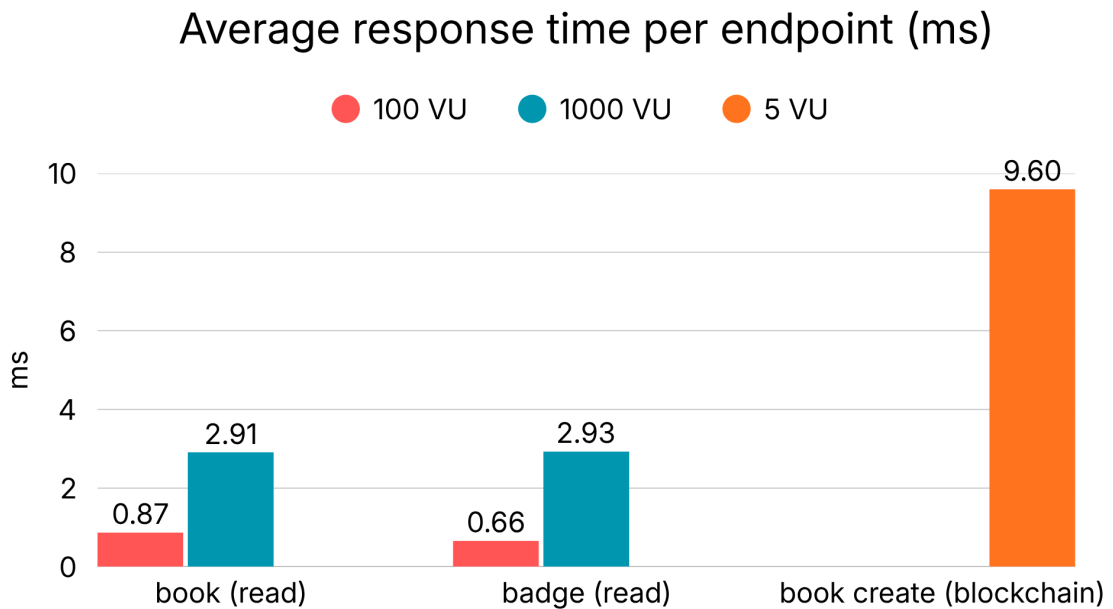
Külön figyelmet kapott az EIP-712 aláírások tesztelése. A funkcionális tesztek ellenőrizték a helyes typed data struktúrával készült aláírás elfogadását, illetve a hibás vagy manipulált aláírások elutasítását. A nonce mechanizmus működése szintén tesztelve lett, így sikeresen megakadályozhatóvá vált ugyanazon aláírás többszöri felhasználása. A replay attack elleni védelem során a rendszer a második azonos aláírással küldött tranzakciót automatikusan visszautasította.

A kitűző rendszer működéséhez kapcsolódóan a mintBadge függvény is tesztelésre került. A vizsgálatok ellenőrizték a sikeres token mintelést, a duplikált kitűzők tiltását és a jogosultságkezelést. Emellett a token URI generálása is validálásra került, biztosítva hogy minden ERC-1155 token a megfelelő metadata fájlra mutasson.

6.3 Backend stresszteszt

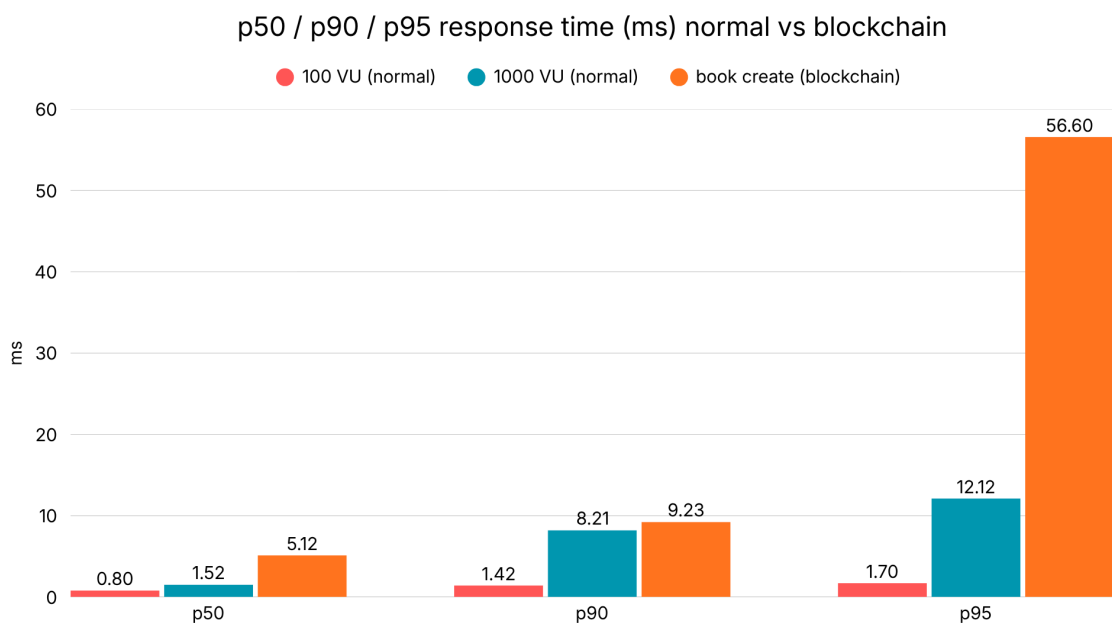
A backend teljesítményének vizsgálatához különböző terhelési szinteken futtattam stresszteszteket. A mérések fő célja annak megfigyelése volt, hogyan változik a válaszidő növekvő egyidejű felhasználószám mellett, illetve mekkora többletterhelést jelent a blokklánc integráció. Az egyszerű lekérdezéseket 100 és 1000 virtuális felhasználóval vizsgáltam, míg a blokkláncot is érintő könyv létrehozás endpoint külön mérésenként szerepelt.

A 6.2-es ábrán látható, hogy a normál olvasási műveletek rendkívül alacsony válaszidővel működnek még nagyobb terhelés alatt is. A könyv és kitűző lekérdezések 1000 egyidejű felhasználó mellett is körülbelül 3 ms körüli átlagos válaszidőt produkáltak. Ezzel szemben a blokkláncot is használó könyv létrehozás endpoint átlagosan 9,60 ms válaszidőt ért el.



6.2 Ábra - Átlagos válaszidő endpointonként

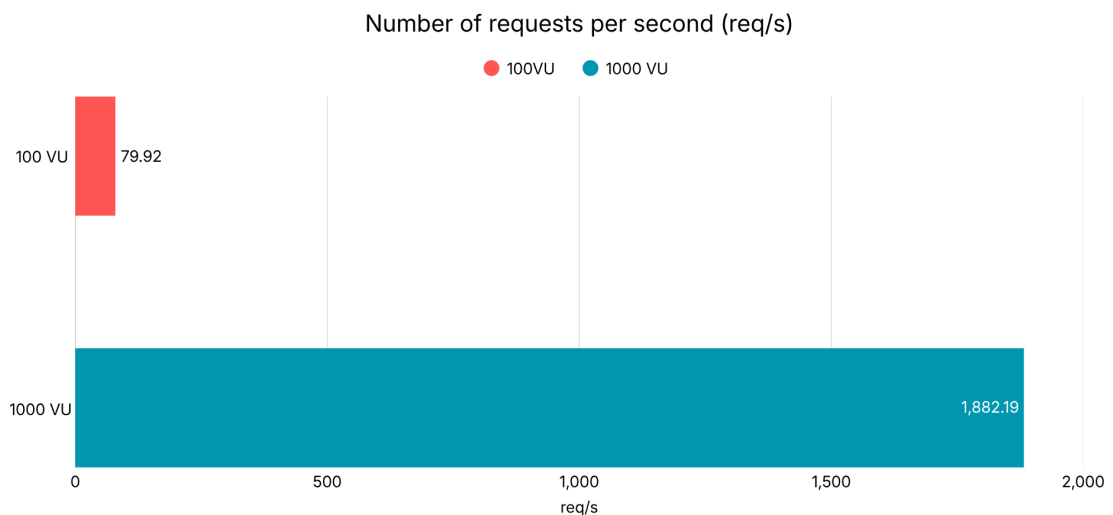
A 6.3-as ábrán vizsgálható mérés a p50, p90 és p95 válaszidőket vizsgálja. Jól látható, hogy a normál endpointok esetében még 1000 virtuális felhasználónál is stabil maradt a rendszer. A p95 válaszidő 12,12 ms volt, ami a 10x-es terhelés növekedés ellenére sem jelentett lineáris lassulást. Ez arra utal, hogy a backend megfelelően skálázódik nagyobb terhelés esetén is.



6.3 Ábra - p50, p90, p95 válaszidő normál és blokklánc műveletek esetében

A blokklánccal kombinált könyv létrehozás endpoint jelentősen lassabb eredményt adott, p95 értéken 56,60 ms körüli válaszidővel. Fontos azonban, hogy maga a HTTP kérés nem várja meg a tranzakció teljes láncra kerülését. A blokklánc hívások aszinkron módon, háttérfolyamatként futnak setImmediate használatával. Ennek hiányában minden kérés több másodperces várakozást eredményezne a tranzakció megerősítéséig.

A 6.4-es ábrán megfigyelhető, hogy a legnagyobb terhelés mellett, 1000 virtuális felhasználó esetén a szerver körülbelül 1882 kérés/másodperc teljesítményt ért el, ami megfelelő eredménynek tekinthető egy olyan rendszer esetében ahol a backend emellett blokklánc kommunikációt is koordinál.



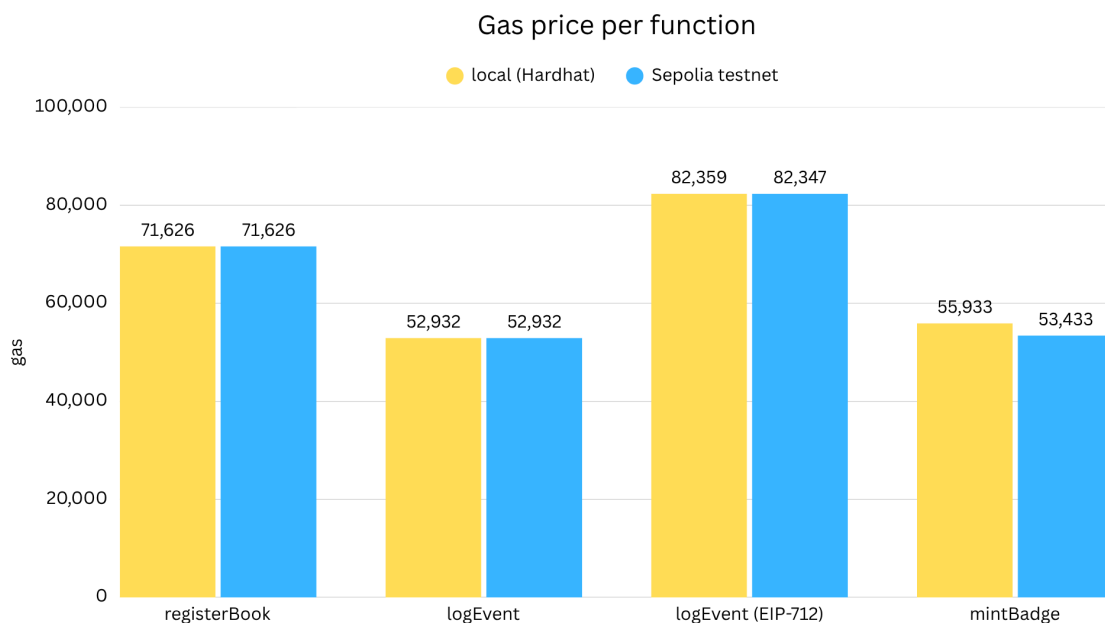
6.4 Ábra - Kérések száma másodpercenként

6.4 Tranzakciós költségek és blokklánc korlátok

A blokklánc alapú működés egyik legfontosabb korlátja a tranzakciók költsége és a megerősítéshez szükséges idő. A tesztelés során külön vizsgáltam az egyes okosszerződés funkciók gas felhasználását lokális Hardhat hálózaton és a Sepolia testneten. A mérés célja annak megfigyelése volt, hogy a különböző műveletek mennyire terhelik a blokkláncot, illetve mekkora többletköltséget jelent az EIP-712 alapú aláírás használata.

A 6.5-ös ábrán látható mérések alapján a logEvent funkció rendelkezett a legalacsonyabb gas igénnyel, körülbelül 52 932 gas felhasználással. A registerBook tranzakció körülbelül 71 ezer gas környékén mozgott, míg a legdrágább művelet az EIP-712 aláírással kombinált logEvent volt, több mint 82 ezer gas értékkel. Ez a többletköltség a digitális aláírás ellenőrzéséből és a nonce kezelésből adódik. A mintBadge funkció viszonylag alacsony költséggel működött, körülbelül 54-56 ezer gas felhasználással.

A lokális Hardhat hálózat és a Sepolia testnet között a gas felhasználásban szinte nem volt különbség. Ez azért történik, mert a gas számítás determinisztikus módon működik, ugyanaz a bytecode és ugyanazok az EVM szabályok kerülnek végrehajtásra mindkét környezetben. A kisebb eltérések főként a storage slot állapotából adódnak.

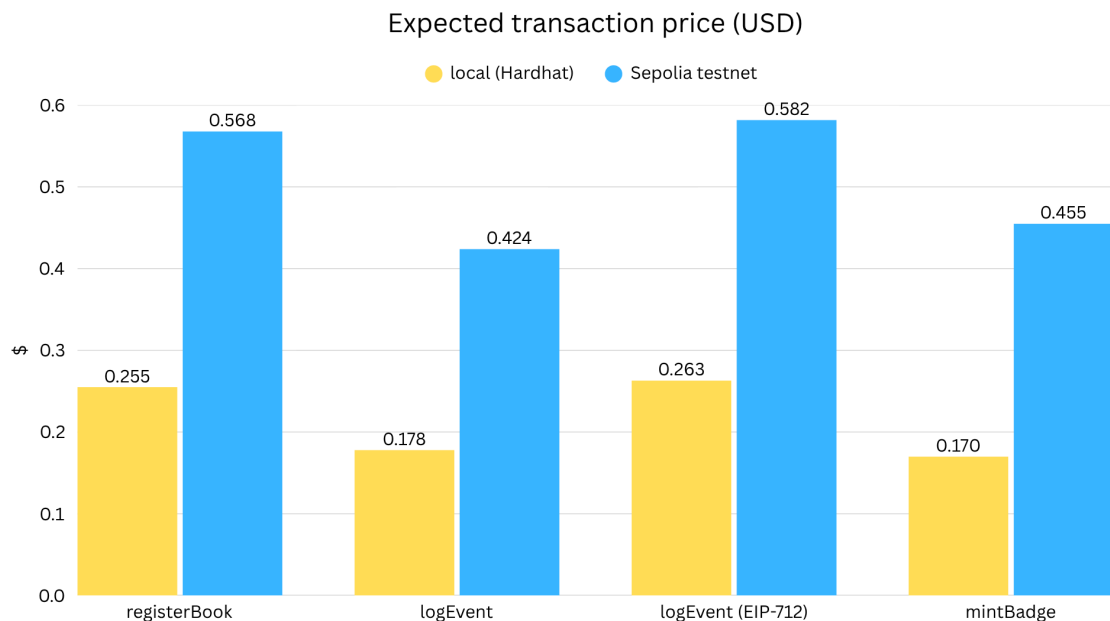


6.5 Ábra - Gas díj funkcióként

A tényleges tranzakciós költségekben azonban már jelentősebb különbség látható. Ahogy a 6.6-os ábrán észrevehető, Sepolia hálózaton minden művelet körülbelül kétszer akkora költséget eredményezett mint a lokális becslések. A registerBook tranzakció például lokálisan körülbelül 0,255 USD költségnek felelt meg, míg Sepolián nagyjából 0,568 USD volt. Hasonló különbség figyelhető meg az összes többi funkciónál is. Az EIP-712 alapú esemény hozzáadás bizonyult a legdrágább műveletnek körülbelül 0,582 USD tranzakciós költséggel.

A tényleges tranzakciós költségek összehasonlításához a mérések során nem valódi fizetés történt, hanem becsült USD értékek kerültek kiszámításra az aktuális

gas price alapján. Bár a Hardhat lokális hálózat és a Sepolia testnet is ingyenesen használható fejlesztői környezet, a tranzakciók ugyanúgy rendelkeznek gas felhasználással és gas árral. A különbség annyi, hogy a testnet ETH nem valódi pénzért kerül felhasználásra.



6.6 Ábra - Tranzakciók ára Hardhat local és Sepolia testnet környezetben

7. Eredmények és értékelés

A fejlesztés során sikerült létrehozni egy olyan webalkalmazást amely ötvözi a hagyományos centralizált rendszerek gyors működését a blokklánc alapú hitelesítéssel. Az alkalmazás képes könyvek és események kezelésére, QR-kód alapú megosztásra, kitűzők kiosztására, valamint az adatok integritásának automatikus ellenőrzésére. A frontend, backend, adatbázis és blokklánc komponensek együttműködése stabilan működik, a rendszer a tesztek alapján nagyobb terhelés mellett is használható maradt.

A projekt egyik legfontosabb eredménye az EIP-712 alapú digitális aláírás sikeres integrációja. A felhasználói események hitelesítése így nem kizárólag a backendre támaszkodik, hanem a felhasználó saját walletje is részt vesz az adat hitelesítésében. Emellett sikerült megvalósítani az automatikus hash alapú verifikációt, amely képes felismerni az adatbázisban történt manipulációkat. A rendszer minden lekérésnél összeveti a blokkláncon tárolt hash értéket a lokálisan generált hashsel, így gyorsan észlelhető bármilyen eltérés.

A koncepció működőképesnek bizonyult, különösen olyan esetekben ahol fontos az adatok eredetének és változatlanóságának bizonyítása. A hagyományos rendszerekben az adatbázis teljes mértékben a szerver bizalmára épül. Ebben a megközelítésben viszont a blokklánc egy külső hitelesítési réteggént működik, amely megnehezíti az adatok észrevétlen módosítását. A felhasználó számára ez átlátható módon jelenik meg a verifikációs állapotokon keresztül.

A hibrid adattárolási modell szintén fontos eredménynek tekinthető. A teljes adatok nem kerülnek a blokkláncra, csak azok hash értékei, így a rendszer megőrzi a gyors működést és az alacsonyabb költségeket. Ez jelentős előnyt biztosít egy teljesen on-chain megoldással szemben, ahol minden adat tárolása rendkívül költséges lenne. A képek, leírások és egyéb metaadatok centralizált tárolása mellett a blokklánc továbbra is képes biztosítani az integritást.

A rendszer legnagyobb korlátját továbbra is a blokklánc tranzakciók sebessége és költsége jelenti. Bár a háttérben aszinkron módon történik a tranzakció kezelés, a láncra kerülés több másodpercet is igénybe vehet. Ettől függetlenül a megoldás jól demonstrálja, hogy a blokklánc technológia nem kizárólag kriptovalutákhoz használható, hanem hagyományos webalkalmazások hitelesítési rétegeként is hatékonyan alkalmazható.

8. Továbbfejlesztési lehetőségek

A rendszer jelenlegi állapotában is teljes értékű webalkalmazásként működik, viszont számos továbbfejlesztési lehetőséget rejt magában. A projekt felépítése lehetővé teszi új funkciók viszonylag egyszerű integrálását anélkül, hogy az alap architektúrát jelentősen módosítani kellene.

Az egyik legérdekesebb továbbfejlesztési irány a könyvek NFT alapú reprezentációja lehetne. Jelenleg a rendszer csak hash értékeket tárol a blokkláncon, azonban a könyvek akár egyedi tokenként is megjelenhetnének. Ez lehetőséget biztosítana digitális tulajdonjog kezelésére, könyv átadásra vagy akár gyűjthető ritka példányok létrehozására is.

A projekt jelenleg böngésző alapú webalkalmazásként működik, viszont mobilalkalmazás formájában még kényelmesebbé válhatna a használata. A QR-kód alapú működés különösen jól illeszkedik mobil környezethez, hiszen a könyvek

történetének megtekintése vagy új események hozzáadása közvetlenül telefonról történhetne.

A képek jelenleg centralizált módon kerülnek tárolásra, emiatt egy további fejlesztési lehetőség decentralizált storage rendszer, például IPFS [30] használata lenne. Ezzel a médiafájlok hitelessége és decentralizáltsága is erősíthető lenne, nem kizárólag a metaadatoké.

Közösségi szempontból szintén tovább bővíthető a rendszer. Több felhasználói interakció, például kommentek, követések, kedvelések vagy közös könyv kihívások jelentősen növelhetnék az aktivitást és a platform közösségi jellegét. A kitűző rendszer szintén továbbfejleszthető szezonális vagy ritka jutalmakkal.

A jelenlegi implementáció legfőbb célja egy stabil alap létrehozása volt, amely demonstrálja hogyan kombinálható egy modern webalkalmazás a blokklánc alapú hitelesítéssel. A projekt architektúrája megfelelő alapot biztosít egy nagyobb, komplexebb platform kialakításához is.

9. Összefoglalás

A szakdolgozat célja egy olyan modern webalkalmazás fejlesztése volt, amely képes ötvözni a hagyományos centralizált rendszerek gyors működését a blokklánc technológia által biztosított hitelességgel és integritásvédelemmel. A fejlesztés során létrejött egy könyvkövető platform, ahol a felhasználók könyveket adhatnak hozzá, eseményeket rögzíthetnek, QR-kódokon keresztül követhetik egy kötet történetét, valamint blokklánc alapú kitűzőket szerezhetnek aktivitásuk alapján.

A projekt során Angular alapú frontend, Node.js backend, PostgreSQL adatbázis és Ethereum kompatibilis okoszerződés került integrálásra egyetlen rendszerbe. A blokklánc nem elsődleges adattárolási réteggént működik, hanem hitelesítési mechanizmusként, amely hash alapú ellenőrzéssel biztosítja az adatok változatlanságát. A megvalósítás során sikeresen integrálásra került az EIP-712 digitális aláírás is, amely lehetővé teszi a felhasználói események kriptográfiai hitelesítését.

A tesztek alapján a rendszer stabil működést mutatott nagyobb terhelés mellett is, miközben a blokklánc integráció ellenére megmaradt az alacsony válaszidő és a megfelelő felhasználói élmény. A fejlesztés bizonyította, hogy a blokklánc technológia

nem kizárólag pénzügyi rendszerekben alkalmazható hatékonyan, hanem hagyományos webalkalmazások kiegészítő hitelesítési rétegeként is jól használható.

A projekt legfontosabb eredménye annak demonstrálása volt, hogy megfelelő architektúrával a decentralizált és centralizált megoldások előnyei kombinálhatók egyetlen rendszerben. A létrejött alkalmazás egyszerre képes gyors működésre, skálázhatóságra és az adatok hitelességének biztosítására.

Irodalomjegyzék

- [1] „carvertical.com/en/about-us”. Elérés: 2026. május 6. [Online]. Elérhető: <https://www.carvertical.com/en/about-us>
- [2] S. Tiwari, „An Introduction to QR Code Technology”, in *2016 International Conference on Information Technology (ICIT)*, dec. 2016, o. 39–44. doi: [10.1109/ICIT.2016.021](https://doi.org/10.1109/ICIT.2016.021).
- [3] R. S. Alsawaier, „The effect of gamification on motivation and engagement”, *The International Journal of Information and Learning Technology*, köt. 35, sz. 1, o. 56–79, 2018.
- [4] J. Zarrin, H. Wen Phang, L. Babu Saheer, és B. Zarrin, „Blockchain for decentralization of internet: prospects, trends, and challenges”, *Cluster Comput*, köt. 24, sz. 4, o. 2841–2866, dec. 2021, doi: [10.1007/s10586-021-03301-8](https://doi.org/10.1007/s10586-021-03301-8).
- [5] B. Preneel, „Cryptographic hash functions”, *Trans Emerging Tel Tech*, köt. 5, sz. 4, o. 431–448, júl. 1994, doi: [10.1002/ett.4460050406](https://doi.org/10.1002/ett.4460050406).
- [6] Q. Wang, R. Li, Q. Wang, és S. Chen, „Non-Fungible Token (NFT): Overview, Evaluation, Opportunities and Challenges”, 2021. október 25., *arXiv*: arXiv:2105.07447. doi: [10.48550/arXiv.2105.07447](https://doi.org/10.48550/arXiv.2105.07447).
- [7] S. Tikhomirov, „Ethereum: State of Knowledge and Research Perspectives”, in *Foundations and Practice of Security*, köt. 10723, A. Imine, J. M. Fernandez, J.-Y. Marion, L. Logrippo, és J. Garcia-Alfaro, Szerk., in *Lecture Notes in Computer Science*, vol. 10723. , Cham: Springer International Publishing, 2018, o. 206–221. doi: [10.1007/978-3-319-75650-9_14](https://doi.org/10.1007/978-3-319-75650-9_14).
- [8] J. D. Drake és J. C. Worsley, *Practical PostgreSQL*. O’Reilly Media, Inc., 2002. Elérés: 2026. május 12. [Online]. Elérhető: https://books.google.com/books?hl=hu&lr=&id=fI1lAgAAQBAJ&oi=fnd&pg=PR4&dq=postgresql+database&ots=a1_M87RYu6&sig=cVRrmx3-Zub_C_4hH7e8iwf0q10
- [9] H. Taherdoost, „Smart contracts in blockchain technology: A critical review”, *Information*, köt. 14, sz. 2, o. 117, 2023.
- [10] C. Dannen, „Solidity Programming”, in *Introducing Ethereum and Solidity*, Berkeley, CA: Apress, 2017, o. 69–88. doi: [10.1007/978-1-4842-2535-6_4](https://doi.org/10.1007/978-1-4842-2535-6_4).
- [11] E. Hildenbrandt és mtsai., „Kevm: A complete formal semantics of the ethereum virtual machine”, in *2018 IEEE 31st Computer Security Foundations Symposium (CSF)*, IEEE, 2018, o. 204–217. Elérés: 2026. május 12. [Online]. Elérhető: <https://ieeexplore.ieee.org/abstract/document/8429306/>

- [12] D. Koutmos, „Network activity and ethereum gas prices”, *Journal of Risk and Financial Management*, köt. 16, sz. 10, o. 431, 2023.
- [13] B. Segendorf, „What is bitcoin”, *Sveriges riksbank economic review*, köt. 2, sz. 1, o. 71–87, 2014.
- [14] D. P. Mishra, S. R. Behera, S. S. Behera, A. R. Patro, és S. R. Salkuti, „Solana blockchain technology: a review”, *International Journal of Informatics and Communication Technology (IJ-ICT)*, köt. 13, sz. 2, o. 197–205, 2024.
- [15] A. H. Sutopo, *Blockchain programming smart contract on polygon*. Topazart, 2023. Elérés: 2026. május 12. [Online]. Elérhető: <https://books.google.com/books?hl=hu&lr=&id=zSG3EAAQBAJ&oi=fnd&pg=PA24&dq=polygon+blockchain&ots=XbB0CLpeQT&sig=nuLGXwruJOoEuZodz8wbSATXn60>
- [16] O. Rodríguez Setó, „Abstracting key management for blockchain services”, 2025, Elérés: 2026. május 12. [Online]. Elérhető: <https://upcommons.upc.edu/entities/publication/65b3f2a3-ca55-4787-b2a4-475714d24651>
- [17] M. Loporchio, D. Di Francesco Maesa, A. Bernasconi, és L. Ricci, „Analyzing ERC-1155 Adoption: A Study of the Multi-token Ecosystem”, in *Complex Networks & Their Applications XIII*, köt. 1187, H. Cherifi, M. Donduran, L. M. Rocha, C. Cherifi, és O. Varol, Szerk., in *Studies in Computational Intelligence*, vol. 1187. , Cham: Springer Nature Switzerland, 2025, o. 385–397. doi: [10.1007/978-3-031-82427-2_32](https://doi.org/10.1007/978-3-031-82427-2_32).
- [18] D. P. Bauer, „ERC-20: Fungible Tokens”, in *Getting Started with Ethereum*, Berkeley, CA: Apress, 2022, o. 17–48. doi: [10.1007/978-1-4842-8045-4_3](https://doi.org/10.1007/978-1-4842-8045-4_3).
- [19] D. P. Bauer, „ERC-721 Nonfungible Tokens”, in *Getting Started with Ethereum*, Berkeley, CA: Apress, 2022, o. 55–74. doi: [10.1007/978-1-4842-8045-4_5](https://doi.org/10.1007/978-1-4842-8045-4_5).
- [20] etherscan.io, „Ethereum (ETH) Blockchain Explorer”, Ethereum (ETH) Blockchain Explorer. Elérés: 2026. május 12. [Online]. Elérhető: <https://etherscan.io/>
- [21] R. A. Achalare, S. V. Patil, és S. S. Patil, „Significance of ISSN and ISBN in Publications”, *Journal of Current Pharma Research*, köt. 5, sz. 1, o. 1378, 2014.
- [22] „What is the Sepolia testnet?”, Alchemy. Elérés: 2026. május 12. [Online]. Elérhető: <https://www.alchemy.com/overviews/sepolia-testnet>
- [23] M. Biehl, *API architecture*, köt. 2. API-University Press, 2015. Elérés: 2026. május 12. [Online]. Elérhető: <https://books.google.com/books?hl=hu&lr=&id=6D64DwAAQBAJ&oi=fnd&pg=PA15&dq=API&ots=zb9N9xMg3z&sig=OqtLYmRgsF0vRgLmu9OZonbCWcU>

- [24] C. Gackenheimer, *Introduction to React*. Apress, 2015. Elérés: 2026. május 12. [Online]. Elérhető: https://books.google.com/books?hl=hu&lr=&id=NZCKCgAAQBAJ&oi=fnd&pg=PR6&dq=react&ots=KCyxVrAC4a&sig=YMD9pdn6klXKyVrdlXaJs8j_dmg
- [25] A. K. Sahani, P. Singh, és V. Jeyamani, „Web development using angular: a case study”, *Journal of Informatics Electrical and Electronics Engineering (JIEEE)*, köt. 1, sz. 2, o. 1–7, 2020.
- [26] „Node.js — About Node.js®”. Elérés: 2026. május 12. [Online]. Elérhető: <https://nodejs.org/en/about>
- [27] V. Surwase, „REST API modeling languages-a developer’s perspective”, *Int. J. Sci. Technol. Eng*, köt. 2, sz. 10, o. 634–637, 2016.
- [28] „Hardhat 3”. Elérés: 2026. május 12. [Online]. Elérhető: <https://hardhat.org>
- [29] „MetaMask Crypto Wallet. Buy and Sell Bitcoin, Ethereum, Solana”. Elérés: 2026. május 12. [Online]. Elérhető: <https://metamask.io>
- [30] T. V. Doan, Y. Psaras, J. Ott, és V. Bajpai, „Toward decentralized cloud storage with IPFS: opportunities, challenges, and future considerations”, *IEEE Internet Computing*, köt. 26, sz. 6, o. 7–15, 2022.

Nyilatkozat

Alulírott Tóth Bence, programtervező-informatikus BSc szakos hallgató, kijelentem, hogy a dolgozatot a Szegedi Tudományegyetem, Informatikai Intézet Szoftverfejlesztési Tanszékén készítettem, programtervező-informatikus diploma megszerzése érdekében.

Kijelentem, hogy a dolgozatot más szakon korábban nem védtem meg, saját munkám eredménye, és csak a hivatkozott forrásokat (szakirodalom, eszközök, stb.) használtam fel. Tudomásul veszem, hogy szakdolgozatomat a Szegedi Tudományegyetem Diplomamunka Repozitóriumban tárolja.

2026. 05. 12. Tóth Bence